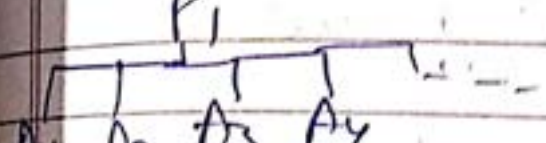
Design & Analysis of Algorithms

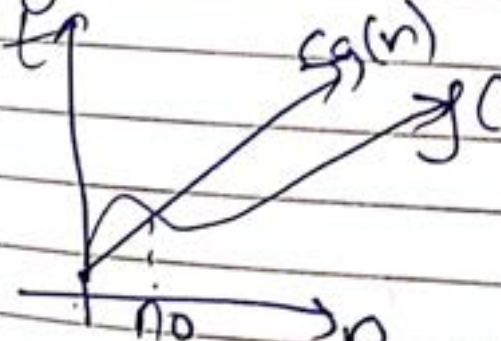
P - Problem
A - Algorithm



(1) Time
(2) Space (Memory)

Notations used — Asymptotic notation

1) Big (oh) O



$f(n) \leq cg(n)$

$n \geq n_0$

c & n are real

$c > 0, n_0 \geq 1$

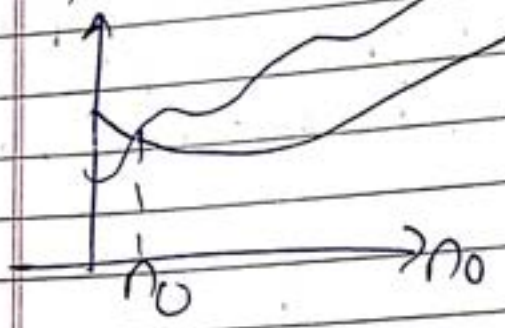
$f(n) = O(g(n))$

Scanned with CamScanner

Eg $f(n)=3n+2$ $g(n)=n$
$f(n)=O(g(n))$ $c=4$
$n_0 \geq 2$

Although $f(n)=O(n^2, n^3, n^n, \ldots 2^n)$
but always take last other bound
$f(n)=O(n)$

2. Big omega Ω

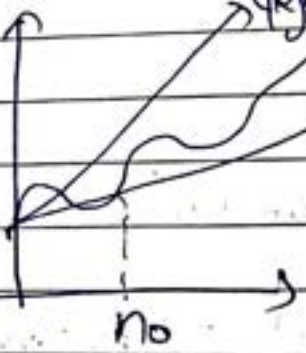


$f(n) \geq cg(n)$
$n \geq n_0$
$c>0, n_0 \geq 1$

$f(n)=3n+2$ $g(n)=n$

$3g(n)=f(n)$ → we take gredent lower bound

3. Big theta Θ



$f(n)=\Theta(g(n))$
$c_1 g(n) \leq f(n) \leq c_2 g(n)$
$c_1, c_2 > 0$
$n \geq n_0, n_0 \geq 1$

$f(n)=3n+2$ $g(n)=n$
$f(n) \leq cg(n)$
$3n+2 \leq 4(n)$, $n_0 \geq 1$
$f(n) \geq cg(n)=n$, $n_0 \geq 1$
$f(n)=g(n)$ $f(n)=\Theta(g(n))$

Scanned with CamScanner

Most used :: we are interested in worst case.

O → worst case
Ω → best case
Θ → average case

[5|7|1|2|4|10|20|11]

linear search

Best case – 1 i.e $\Omega(1)$
Worst case – n i.e $O(n)$
Avg case – $\frac{n}{2}$ i.e $\Theta(\frac{n}{2}) = \Theta(n)$

Algo

Iterative (loops) = Recursive (same function called again and again)

both are equivalent & interconvertible

If no iteration or recursion constant running time

```
A() {
int i,j;
for(i=1 to n)
for(j=1 to n)
printf("avi");
}
```

$O(n^2)$

```
A() {
i=1, s=1;
while(s<=n){
i++;
s=s+i;
printf("avi");
}
```

s	1	3	6	10	15	21	...
i	1	2	3	4	5	6	...

→ $O(\sqrt{n})$

Scanned with CamScanner

Q
```
A() {
  int i, j, k, n;
  for (i=1; i<=n; i++) {
    for (j=1; j<=i; j++) {
      for (k=1; k<=100; k++)
        {printf("Ravi");
}
```

i=1	i=2	i=3
j=1 time	j=2 times	j=3
k=100 time	k=2*100	k=3*100 =300
i=4	i=5	i=n.
j=4 times	j=5 times	j=n. time
k=4x100	k=5x100.	k=nx100

Total add all K simply $100 + 2\times100 + 3\times100 + \dots + n\times100$

$100\left(\frac{n(n+1)}{2}\right) = O(n^2)$

Q
```
A(){
  int i, j, k, n;
  for (i=1; i<=n; i++){
    for (j=1; j<=i^2; j++) {
      for (k=1; k<=n/2; k++){
        printf("Ravi");
}
```

i=1	i=2
j=1	j=4 times → same
k=n*1/2	k=n*4/2 for 2nd

Scanned with CamScanner

Date. 2.11
Page.

But simple way $i \to n$ (worst case)
$j \to n^2$ (worst case)
$k \to \frac{n}{2}$

$O(n^4)$

Q A) C
```
for(i=1; i<n; i=i*2)
    printf("Hai");
```
$i = 1, 2, 4, \ldots \to \log_2 n$

$2^0 \; 2^1 \; 2^2 \; \ldots \; 2^k$ $\quad 2^k = n$

$k = \log_2 n$

if $i*3$ $\log_3 n$
$i*6$ $\log_6 n$
$i*10$ $\log_{10} n$

Q A) C
```
int i, j, k;
for(i=n/2; i<=n; i++)
    for(j=1; j<=n/2; j++)
        for(k=1; k<n; k=k*2)
            printf("Hai");
```
Ans $O(n^2 \log n)$

Scanned with CamScanner

→ A(){
int i, j, k;
for(i=n/2; i<=n; i++)
for(j=1; j<=n; j=2*j)
for(k=1; k<=n; k=k*2)
printf("ravi");
}

$O(n(\log_2 n)^2)$

→ A(){
while(n>1){ $\log_2 n$
$n = \frac{n}{2}$ $O(\lfloor \log_2 n \rfloor)$
}}

→ A(){
for(i=1; i<=n; i++)
for(j=1; j<=n; j=j+i)
printf("ravi");
}

~~n*n = O(n^2)~~ Tricky ques

i=1	i=2	i=3	...	i=n
j=1 to n	j=1 to n	j=1 to n	...	j=1 to n
n times	$\frac{n}{2}$ times	$\frac{n}{3}$ times	...	n/n

$n\left(1 + \frac{1}{2} + \frac{1}{3} \cdots + \frac{1}{n}\right) = n\log n$
$= O(n\log n)$

Scanned with CamScanner

```
A(){                                  k
int n=2^k;
for(i=1; i<=n; i++){
    j=2;
    while(j<=n){
        j=j^2
        printf("hai");
    }
}
}
```

k=1	k=2	k=3	k=k
n=4	n=16	n=64=2^8	
j=2,4	j=2,4,16	j=2,2^2,2^4,2^8	
n×2 times	n×3 times	n×4 times	n*(k+1)

So $n*(k+1)$

n^{2^k}

$\log_2 n = 2^k$

$k = \log\log n$

$n(\log\log n + 1)$

$O(n \log\log n)$

Recursive

```
A(n){
if(n>1)          Decided by anchor base condition
return (A(n-1))
}
```

$T(n) = 1 + T(n-1)$

Back substitution:

$T(n-1) = 1 + T(n-2)$

$T(n-2) = 1 + T(n-3)$

$T(2) = 1 + T(1)$

$T(n-1) = n-1+T(1)$

$T(n-1) = O(n)$

Scanned with CamScanner

$T(n) = n + T(n-1)$; $n > 1$
$= 1$; $n = 1$

$T(n) = n + T(n-1)$
$T(n-1) = n-1 + T(n-2)$

$T(2) = 2 + T(1)$

$T(n) = 2 + 3 + 4 + 5 + \cdots + n$
$T(n) = O(n^2)$

Recursion Tree Method is effective for some problems

$T(n) = 2T(n/2) + C$; $n > 1$
$= C$; $n = 1$

$T(n)$ — C — C
$T(n/2)$ $T(n/2)$ — $2C$
$T(n/4)$ $T(n/4)$ $T(n/4)$ $T(n/4)$ — $4C$
— $8C$

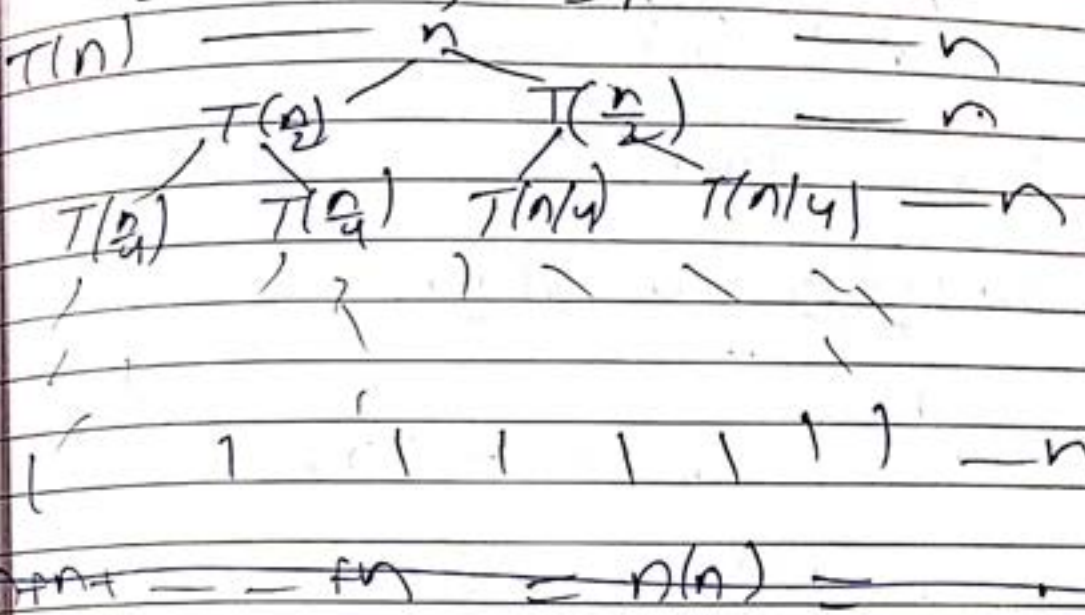
$T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ — nC

$C + 2C + 4C + \cdots + nC$
$C(1 + 2 + 4 + \cdots + n)$ $n = 2^k$
$= C(2^{k+1} - 1) = O(n)$

Date / Page 215

$T(n) = 2T(\frac{n}{2}) + n, \; n>1$

$= 1 \quad , n=1$



$\frac{n}{2^0} - \frac{n}{2^1} - \frac{n}{2^2} - \frac{n}{2^3} - \cdots - \frac{n}{2^k}$

$n = 2^k, \quad k = \log n$

$n(\log n + 1)$

Master theorem

2^n n^2 Which bigger.

When this kind of problem do

1) try to substitute 4 in other like n^4, n^3

2) take log & substitute large values of n to check

$n \log 2 = 2 \log n$

$n = 100$

$100 \quad 100^2$

$100 \log 2 > 2 \log 100$ So cleared

$2^n > n^2$

Scanned with CamScanner

→ n^2 $n\log n$

n $\log n$

$n > \log n$

→ n $\log n/100$

$\log n > 100 \log\log n$

$n = 2^{128}$

$128 < 100 \times 7 = 700$

$n = 2^{1024}$

$1024 > 1000$

→ $n^{\log n}$ $n\log n$

$\log n \log n$ $\log n + \log\log n$

$n = 2^{1024}$

1024×1024 $1024 + 10$

$n = 2^{2^{20}}$

$2^{20} \times 2^{20}$ 2^{20+20}

$n\log n > n\log n$

→ $\sqrt{\log n}$ $\log\log n$

$\frac{1}{2}\log\log n$ $\log\log\log n$

$n = 2^{2^{10}}$

5 → 3.5

Scanned with CamScanner

$n^{\log n}$ $n^{\log n}$

$n \log n$ $\log n \log n$

$\frac{1}{2} \log n$ $\log \log n$

$n = 2^{128}$

$64 > 7$

$$f(n) = \begin{cases} n^3 & 10 \le n < 10{,}000 \\ n^2 & n \ge 10000 \end{cases}$$

$$g(n) = \begin{cases} n & 0 < n \le 100 \\ n^3 & n > 100 \end{cases}$$

	0–99	100–9999	10,000 — ∞
$f(n)$	n^3	n^3	n^2
$g(n)$	n	n^3	n^3

So $f(n) = O(g(n))$ $n_0 = 10{,}000$

$f_1 = 2^n$, $f_2 = n^{3/2}$, $f_3 = n \log n$, $f_4 = n^{\log n}$

2^n $n^{3/2}$

$n \log 2$ $\frac{3}{2} \log n$

n $\frac{3}{2} \log n$

$2^{128} > \frac{3}{2} \times 128$

2^n $n \log n$

n $\log n \log n$

$2^{128} > 128 \times 128$

f_1 largest

2^n $n \log n$

n $\log n + \log \log n$

$2^{128} > 128 + 7$

$n^{\log n}$ $n^{3/2}$

$\log n > \frac{3}{2}$ $(f_4 > f_2)$

$n^{\log n}$ $n \log n$

$\log \log n$ $\log n + \log \log n$

$128 \times 28 > 128 + 7$

f_4 2nd largest

$n^{3/2}$ $n \log n$

$\frac{3}{2} \log n > \log n + \log \log n$

$f_1 > f_4 > f_2 > f_3$

Scanned with CamScanner

$(\log n)^2$ $\log^2 n$
$\log n * \log n$ $\log \log n$

Master Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + \Theta(n^k \log^p n)$$

$a \geq 1,\ b > 1,\ k \geq 0$ & p is real

1) if $a > b^k$, then $T(n) = \Theta(n^{\log_b a})$

2) if $a = b^k$
   a) if $p > -1$ then $T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$
   b) if $p = -1$ then $T(n) = \Theta(n^{\log_b a} \log \log n)$
   c) if $p < -1$ then $T(n) = \Theta(n^{\log_b a})$

3) if $a < b^k$
   a) if $p \geq 0$ $T(n) = \Theta(n^k \log^p n)$
   b) if $p < 0$ $T(n) = O(n^k)$

Q-1 $T(n) = 3T\left(\frac{n}{2}\right) + n^2$

$a = 3,\ b = 2,\ k = 2,\ p = 0$

$a < b^k$

$p = 0$ $T(n) = \Theta(n^k \log^p n)$

$= \Theta(n^2)$

Q) $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

$a = 4,\ b = 2,\ k = 2,\ p = 0$

$a = b^k$ $p = 0$ i.e. > -1

$T(n) = \Theta(n^{\log_2 4} \log^0 n)$

$= \Theta(n^2 \log n)$

Scanned with CamScanner

Date
Page 219.

→ $T(n) = T(\frac{n}{2}) + n^2$

$a=1, b=2, k=2, p=0$

$1 < 2^2$

$p=0$

$T(n) = \theta(n^2 \log^0 n) = \theta(n^2)$

→ $T(n) = 2^n T(\frac{n}{2}) + n^n$ × Not applicable

→ $T(n) = 16\,T(\frac{n}{4}) + n$

$a=16, b=4, k=1, p=0$

$16 > 4$

$T(n) = \theta(n^{\log_b a}) = \theta(n^2)$

→ $T(n) = 2T(\frac{n}{2}) + n\log n$

$a=2, b=2, k=1, p=1$

$2^1 = 2^1, p=1 \quad p > -1$

$\theta(n^{\log_b a} \log^{p+1} n) = \theta(n \log^2 n)$

→ $T(n) = 2T(\frac{n}{2}) + \frac{n}{\log n}$

$a=2, b=2, k=1, p=-1$

$2 = 2^1$

$p=-1$

$\theta(n^{\log_b a} \log\log n) = \theta(n \log\log n)$

→ $T(n) = 2T(\frac{n}{4}) + n^{0.51}$

$a=2, b=4, k=0.51, p=0$

$a < b^k$

$\theta(n^k \log^p n) = \theta(n^{0.51})$

Scanned with CamScanner

a) $T(n) = 0.5(T(n/2)) + i^n$
$a = 0.5$ ✗ not applicable

b) $T(n) = 6T(\frac{n}{3}) + n^2 \log n$
$a = 6 \quad b = 3 \quad k = 2 \quad p = 1$
$6 < 3^2$
$T(n) = \Theta(n^k \log^p n)$
$= \Theta(n^2 \log n)$

h) $T(n) = 64(T(\frac{n}{3})) - n^2 \log n$
✗ Not valid

12) $T(n) = 7T(n/3) + n^2$
$a = 7, \quad b = 3 \quad k = 2 \quad p = 0$
$a < b^k$
$\Theta(n^k) = \Theta(n^2)$

13) $T(n) = 4T(\frac{n}{2}) + \log n$
$a = 4, \quad b = 2, \quad k = 0, \quad p = 1$
$4 > 2^0$
$T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$

14) $T(n) = \sqrt{2}\, T(n/2) + \log n$
$a = \sqrt{2} \quad b = 2 \quad k = 0 \quad p = 1$
$\sqrt{2} > 2^0$
$T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 \sqrt{2}})$
$= \Theta(\sqrt{n})$

Scanned with CamScanner

(5) $T(n) = 2T(n/2) + \sqrt{n}$

$a = 2, \; b = 2, \; k = 1/2, \; p = 0$

$2 > 2^{1/2}$

$T(n) = \Theta(n^{\log_2 2})$

$= \Theta(n)$

(6) $T(n) = 3T(n/2) + n$

$a = 3, \; b = 2 \; k = 1$

$3 > 2^1$

$T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{\log_2 3})$

(7) $T(n) = 3T(n/3) + \sqrt{n}$

$a = 3, \; b = 3, \; k = 1/2, \; p = 0$

$3 > 3^{1/2}$

$T(n) = \Theta(n^{\log_3 3}) = \Theta(n^1)$

(8) $T(n) = 4T(n/2) + cn$

$a = 4, \; b = 2, \; k = 1, \; p = 0$

$T(n) = \Theta(n^2)$

$T(n) = 3T\left(\frac{n}{4}\right) + (n \log n)$

$a = 3, \; b = 4, \; k = 1, \; p = 1$

$3 < 4$

$T(n) = \Theta(n \log n$

Scanned with CamScanner

Space complexity
We say size of input n

n → O(n)
- O(1)
- O(n²)

Iterative

→ Algo (A, 1, n){ (A: array)
int i; — O(1)
for(i=1 to n)
A[i]=0;
}

O(1) because n is provided from question itself.

→ Algo (A, 1, n){
int i;
create B[i]; — O(n)
for(i=1 to n)
B[i]=A[i];
}

→ Algo(A, 1, n){
create B[n,n];
int i,j;
for(i=1 to n) O(n²)
for(j=1 to n)
B[i,j]=A[i];
}

Scanned with CamScanner

→ Head recursion
→ Tail
→ Body ;;

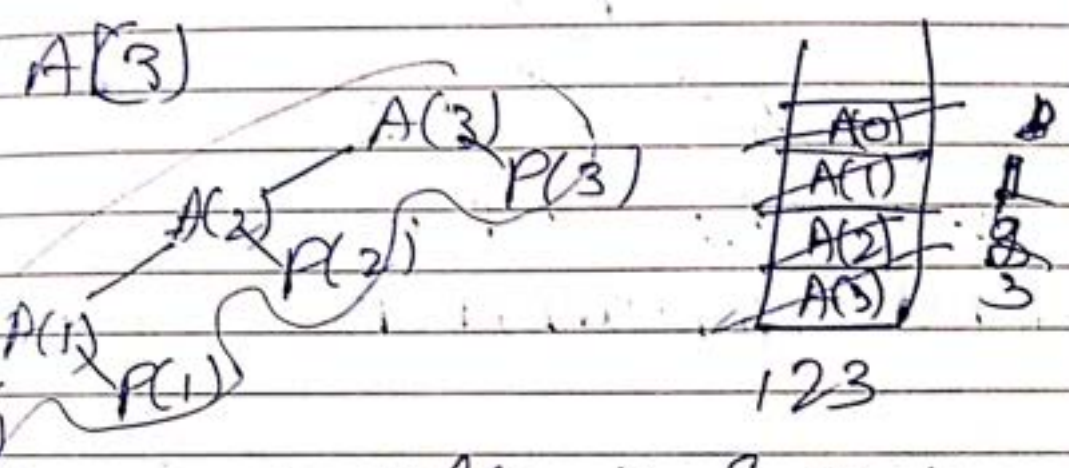
Date
Page 223

Recursive version

```
A(n) {
if (n>=1) {
    A(n-1);   //Head recursion.
    Printf(n);
}}
```

To visualise smell use tree if stack

let's say A(3)



for 3 recursive calls - 4 So n+1

Traverse Top to Down Left to Right.

for n → n+1 Recursive calls and space complexity (n+1) K → size of call of stack

So O(Kn) = O(n)

Time complexity $T(n) = T(n-1) + 1$

$T(n-1) = T(n-2) + 1$

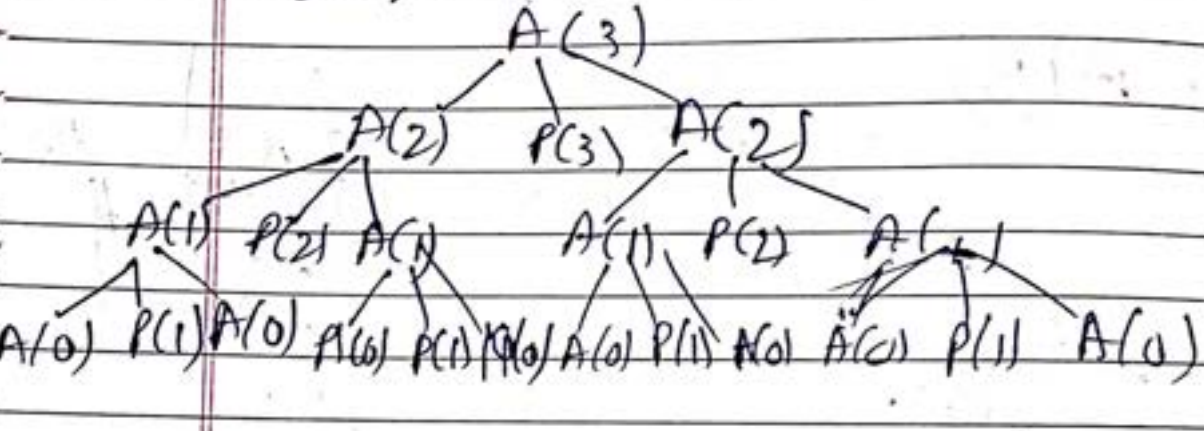
$T(\quad) = T(0) + 1$

$T(n) = O(n)$

Scanned with CamScanner

Q
```
A(n){
if(n>=1)
{A(n-1);
 f(n);
 A(n-1);}
}
```

Ans Let's go for A(3)



A(3) = 15 recursive calls $= 2^{3+1} - 1$

A(2) = 7 " " " $= 2^{2+1} - 1$

A(1) = 3 " " " $= 2^{1+1} - 1$

$A(n) = 2^{n+1} - 1$



Space = 4

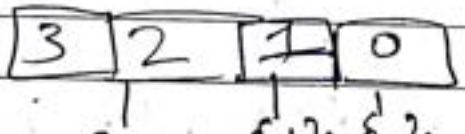
So $O(n+1) = O(n)$

[2,3,1,2,]

123 12 | 3 | 12
A(2) A(2)

A(2) is repeated again, so instead of performing A(2) for 2 times, we prefer to store the result somewhere else. This process is called dynamic programing

In above Q



DP helps in reducing time complexity

Time complexity

$T(n) = T(n-1) + 1 + T(n-1)$

$T(n) = 2T(n-1) + 1$

$T(n-1) = 2T(n-2) + 1$

$T(n-2) = 2T(n-3) + 1$

using back substitution

$T(n) = 2(2T(n-2)+1) + 1$

$= 2^2 T(n-2) + 2 + 1$

$= 2^2 (2(T(n-3)+1) + 2 + 1$

$= 2^3 T(n-3) + 2^2 + 2 + 1$

$= 2^k T(n-k) + 2^{k-1} + \dots + 2^2 + 2 + 1$

$k = n$

$= 2^n T(0) + 2^{n-1} + \dots + 2^2 + 2 + 1$

$= 2^n + 2^{n-1} - \dots$

$= O(2^{n+1})$

$= O(2^n)$

Scanned with CamScanner

25/9/20

Amortized Analysis

Sometimes when we perform any operation on a algo it proves out to be very costly but when same operation performed again then not.

In such cases instead of computing cost/each we perform # of operations and then find average (amortized cost).

→ In an amortized analysis, we average the time required to perform a sequence of data structures operations over all the operations performed.

→ Using an amortized analysis, we can show that the average cost of an operation is small, even though a single operation within the sequence might be expensive.

→ Used to analyze time complexities of hash tables, splay trees & disjoint sets

→ Techniques used in amortized analysis
1) Aggregate Method
2) Accounting
3) Potential Method

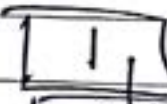
$(T_1 + T_2 + T_3 + \cdots + T_n)/n$

Date | Page | 227

Aggregate Analysis –

Hash table insertions (Increase the size of table whenever it becomes full) –
→ Allocate memory for a large table of size double the old table
→ Copy the contents of old table to new table.
→ Free the old table.

Eg At start 1 elem



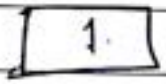
If 2 arrives then



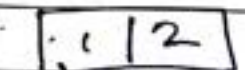
etc.

Details –

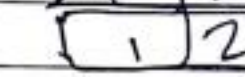
Initially table is empty & size is 0

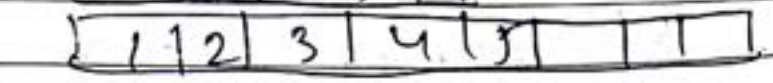
Insert 1 

Overflow

Insert 2 (overflow) 

Insert 3 (,,) 

Insert 4 (,,) 

Insert 5 

Item #	1	2	3	4	5	6	7	8	9	10
Table Size	1	2	4	4	8	8	8	8	16	16
Cost	1	2	3	1	5	1	1	1	9	1

2 moves 2 inserts

4 moves 2 inserts

8 moves 4 inserts

Scanned with CamScanner

Amortized cost = $\frac{(1+2+3+1+5+1+1+1+9+\ldots)}{n}$ ($1+1$, $1+2$, $1+4$)

n times, $\log(n-1)+1$ times

$= \frac{(1+1+1+1\ldots 11)+(1+2+4+8+\ldots)}{n}$

$= \frac{n+2n}{n}$

$= O(1)$

Bubble Sort

```
BubbleSort(int a[], n){
  int swapped, i, j;
for(i=0; i<n; i++){
  int swapped=0;
   for(j=i, j<n; ++j){
   if(a[j+1]<a[j]){
    swap(a[j+1], a[j]);
    swapped=1;
   }
  if(swapped==0)
    break;
}
```

Time complexity = $O(n^2)$ — worst case

Best Case — $O(n)$

Note → At end of each iteration largest elements accomodate at right end

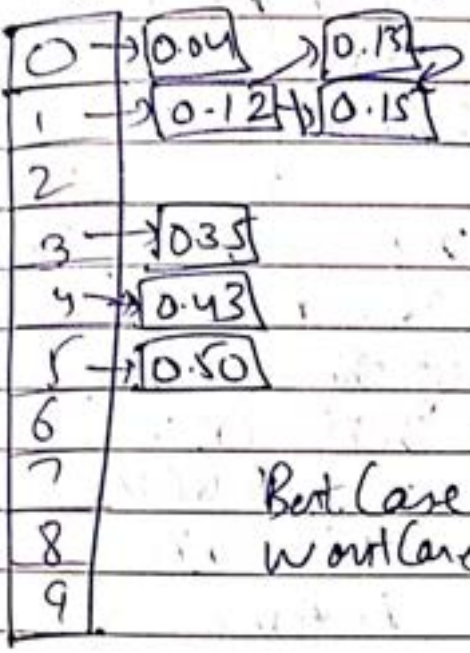
Bucket Sort →

Page 229

a large set of floating point numbers which are in the range 0.0 to 1.0 are uniformly distributed across the range.

0.35, 0.12, 0.43, 0.15, 0.04, 0.50, 0.132

Visually we have few buckets (let's say of size 10) so (Think of it as hash table & Chain)



So
0.04
0.12
0.132
0.15
0.35
0.43
0.80

Time Step
Best Case O(n)
Worst Case O(n²)

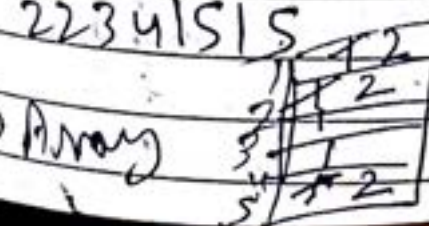
Now all the numbers having 0 after decimal will point to 0. 0 & so on

0.15 when arrives it is compared with 0.12 if it is > then put behind if < put ahead

Counting sort → ~~Assumption~~ Assumption - Keys are always in particular range

e.g. (1-5) of K

2 2 3 4 1 5 1 5



Traverse this array now - 1 1 2 2 3 4 5 5

Time O(n+k)
Space O(k)

Scanned with CamScanner

Disadv:-
1) Range required
2) If range is large Time + Space both are very high irrespective of array size
eg 1, 1000, 2^3

Heap Intro-1

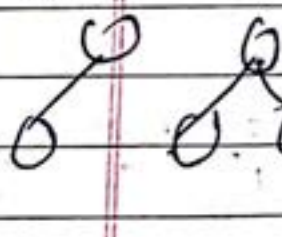
	insert	search	find min	Delete min
Unsorted array	O(1)	O(n)	O(n)	O(n)
Sorted array (Increasing order)	O(n)	O(log n)	O(1)	O(n)
Linked list	O(1)	O(n)	O(n)	O(n)

insert find min delete min are not optimized for any of these above
eg unsorted O(1) O(n) O(n) (when)
sorted O(n) O(1) O(n) (when)

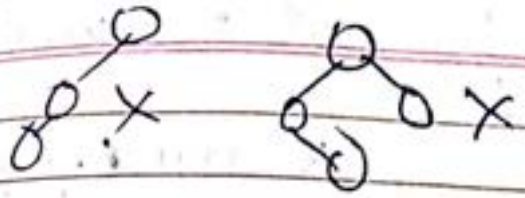
So we use heap

Min Heap	O(log n)		O(1)	O(log n)

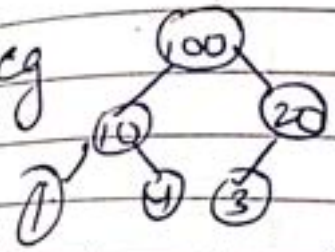
Heap is a binary tree or 3-ary - n-ary
Let's take binary tree
Every heap is almost complete binary tree

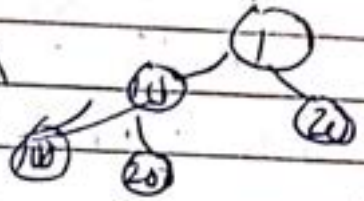
eg 
and filled from left to right
Don't go to next level before filling previous one

Scanned with CamScanner

eg 

Max heap → root always maximum.

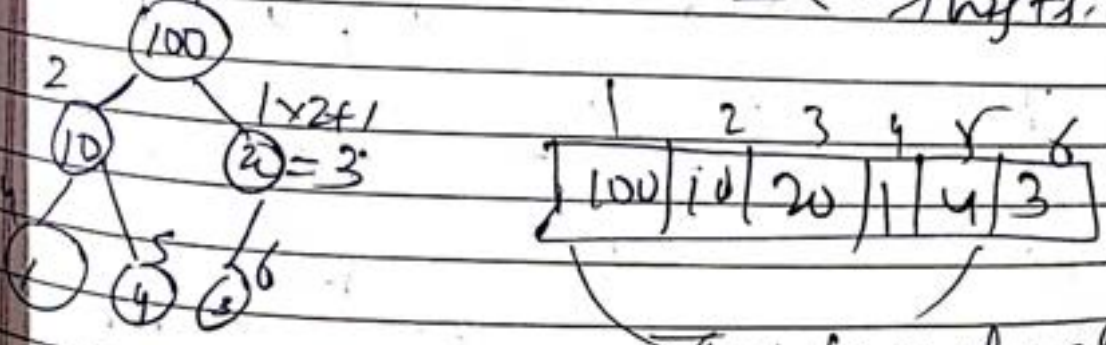
eg 

Min heap → root always min 

Even though it looks like a tree DS, but we are going to put all elements in an array & traverse it.

and index will be $2i$ (left), $2*i+1$ (right)

and from child parent = $\frac{i}{2}$ (Basically left, right shifts. etc.)



That's why almost complete, If tree not almost complete then empty spaces in array

$L(i) = 2i$ → left shift

$R(i) = 2i+1$ → left shift + add 1

$P(i) = i/2$ → right shift

Scanned with CamScanner

Tell if max heap. If not fill which point it is head

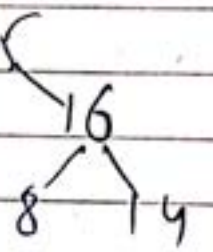
	A.length	A.heapsize
1) 25, 12, 6, 13, 10, 8, 14	7	1 (:) 12
2) 25, 14, 16, 13, 10, 8, 12	7	7 (14<16)
3) 25, 14, 13, 16, 10, 8, 12	7	1
4) 25, 14, 12, 13, 10, 8, 16	7	2 (12<16)
5) 25, 14, 13, 12, 10, 8	5	5
6) 14, 12, 13, 8, 10	5	5
7) 14, 13, 8, 12, 10	5	5
8) 14, 13, 12, 8, 10	5	5
9) 89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70	13	2

10

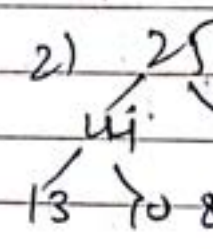
1)



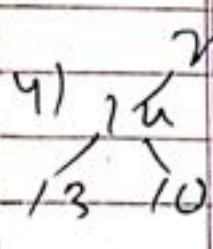
2)



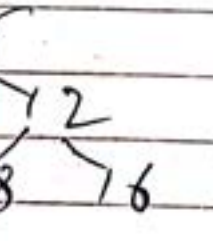
3)



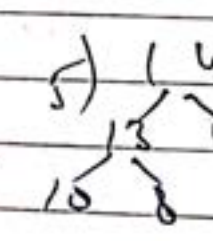
4)



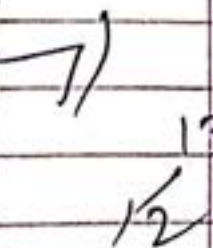
5)

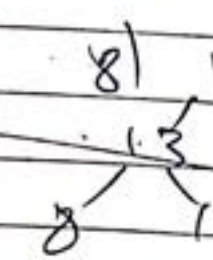


6)



7)





Given set of its many max heap possible

P.S. — 14, 13, 12, 10, 8 — Max heap
8, 10, 12, 13, 14 — Min heap
Any element in descending order is max heap
ascending

Scanned with CamScanner

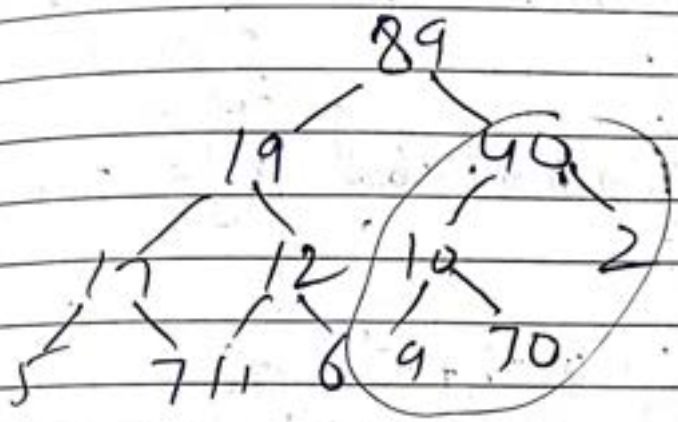
Date
Page 233

→ Best time complexity

Also to sort o(n log n)

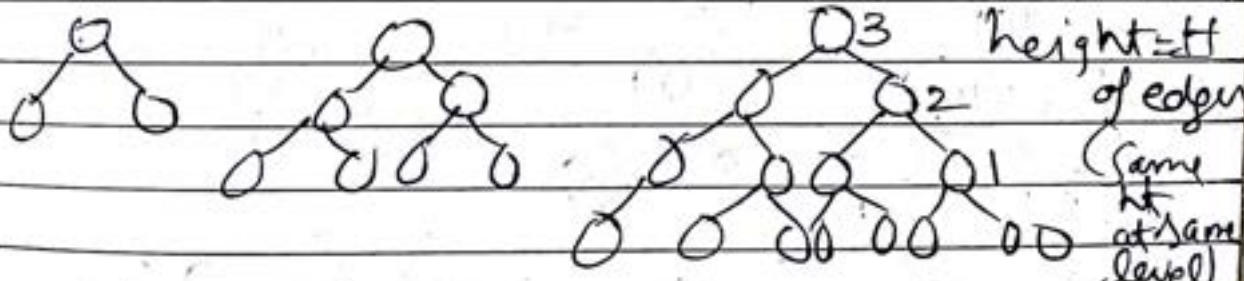
→ So this time to make max/min heap (not sorted)

∴ we can construct in o(n) but to don't sort we will see in next lectures.



→ 22, 40 contains 70

There are some properties of complete binary trees—



Height overall = Height of root

Height	1	2	3	4	h	log2n
Nodes	3	7	15	31	$2^{h+1}-1$	n

$3^{h+1}-1$ complete ternary tree

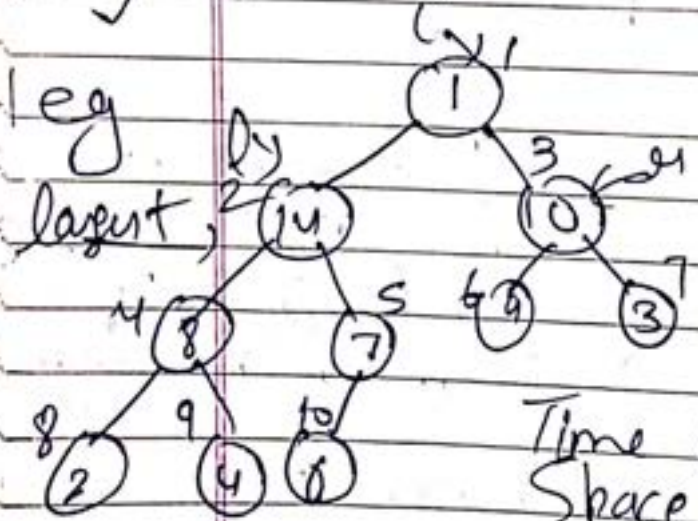
→ take floor for ternary log3n

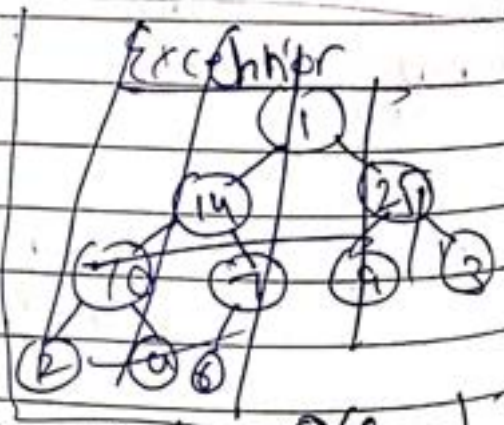
Convert Array to Max Heap → We are not sorting

```
MAX_HEAPIFY(A, i){
l = 2i;
r = 2i + 1;
if l ≤ A.heap_size and A[l] > A[i]
    largest = l;
else largest = i;
if (r ≤ A.heap_size and A[r] > A[largest])
    largest = r;
if (largest ≠ i)
    exchange A[i] with A[largest]
    MAX_HEAPIFY(A, largest)
}
```

If complete (smart) binary tree, leaf start from $(\lfloor \frac{n}{2} \rfloor + 1$ to $n)$

eg $\frac{6}{2} + 1 = 4$





Time complexity $O(\log n)$

Space : $O(\log n)$ stack

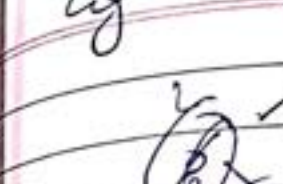
How to Build a heap

```
BUILDMAX_HEAP(A){
A.heap_size = A.length
for (i = ⌊A.length/2⌋ down to 1)
    MAX_HEAPIFY(A, i)
}
```

$\lfloor \frac{n}{2} \rfloor + 1$ to n excluded

Scanned with CamScanner

4g 9 6 5 0 8 2 7 [2.35]



$\lfloor \frac{9}{2} \rfloor + 1 = 5$

So 5 onwards leaf

Traverse 4 → swap 4 & 9 index
3 → " 3 & 7 " "
2 → " 5 & 4 " " etc.

~~Just Anotherway~~

We are going from $\lfloor \frac{n}{2} \rfloor$ to 1 so n.

And height of tree i.e MAX-heap = $\log_2 n$

So Time Complexity $O(n \log n)$ ✗

let's see why wrong this way



leaf $\lfloor \frac{n}{2^{h+1}} \rfloor = \lceil \frac{15}{8} \rceil = 8$ (height 0)

$\frac{n}{2^2} = \lceil \frac{15}{4} \rceil = 4$

height 1

height 2 = 2, height 3 = 1

Total time —

$$\sum_{0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = \sum_{0}^{\log n} \left\lceil \frac{n}{2^{h}2} \right\rceil O(h)$$

$$= \frac{cn}{2} \sum_{h=0}^{\log n} \left\lceil \frac{h}{2^h} \right\rceil = O\left(\frac{cn}{2} \sum_{h=0}^{\infty} \frac{h}{2^h}\right)$$

HP (2 by today)

$= O\left(\frac{cn}{2}(2)\right) = O(n)$

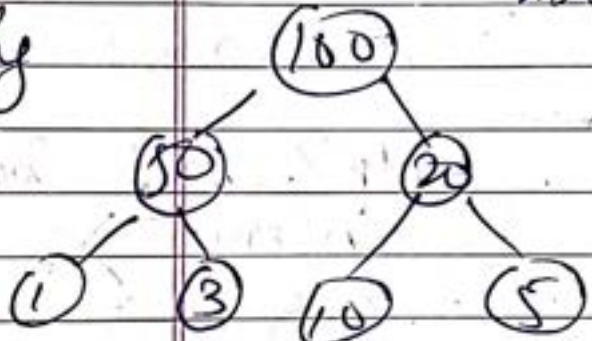
Basically — $O(\log n)$

Scanned with CamScanner

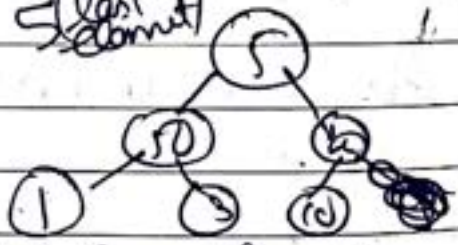
Delete Max from Max heap

```
Heap_Extract_max (A)
{
if (A.heap_size < 1)
    error "heap underflow"
max = A[1]
A[1] = A[A.heap_size]
A.heap_size = A.heap_size - 1
MAX_HEAPIFY (A, 1)
return max
}
```

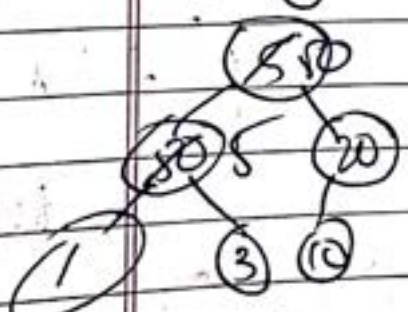
Array 100 50 20 1 3 10 5



What we do is delete 100 & replace it by last element



Now apply MAX_HEAP function, so



Time req - time for MaxHEAPify = O(log n)

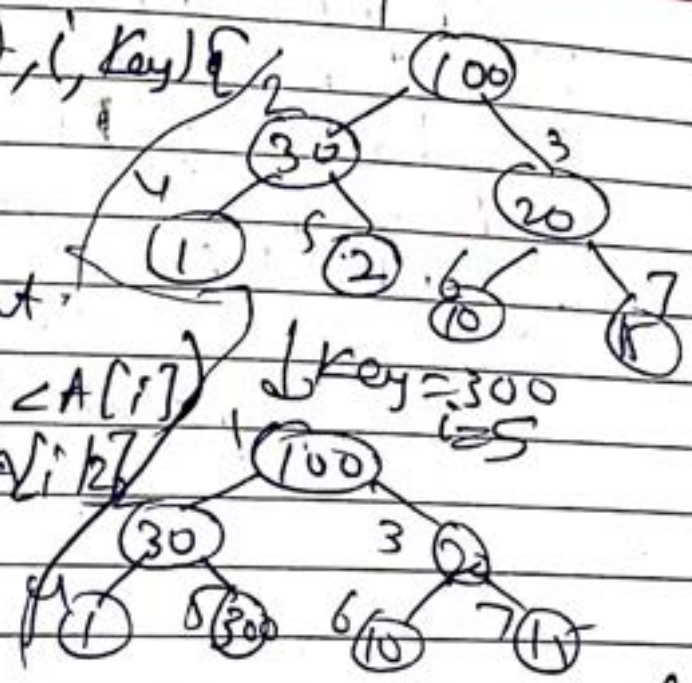
Space complexity = O(log n)

Scanned with CamScanner

1. Increase the value of node

```
Heap_increase_key (A, i, key){
if (key < A[i])
    error
A[i] = key
while (i > 1 and A[i/2] < A[i])
    exchange A[i] & A[i/2]
    i = i/2
}
```

Time = O(log n)
Space = O(1)



so we check 300 with parent and so on

Decreasing a key should call MaxHeapify itself (O log n)

2. Insert an element.

1) Insert element at last index
2) Then swap with parent if > parent and so on.
   (or modify a bit increase key function)

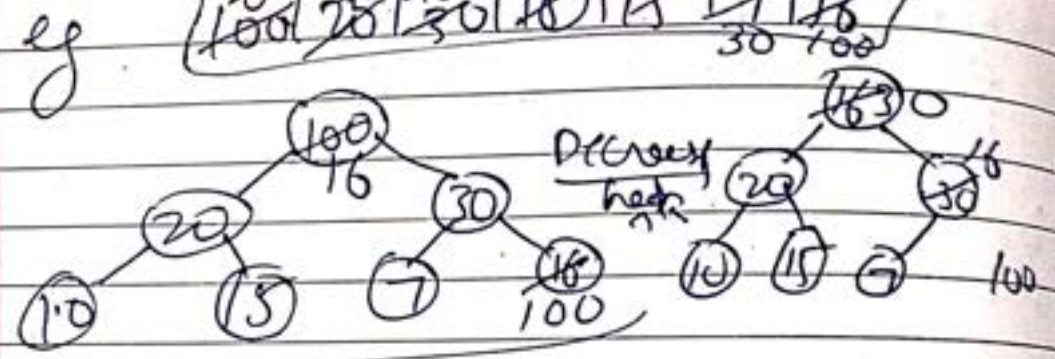
i.e insert (-∞) then pass key

Maximum	findMax O(1)	deleteMax O(log n)	insert O(log n)	Key O(log n)	Delete O(log n)
	findmin O(n)	Recalculation O(n)		Search (random) O(n)

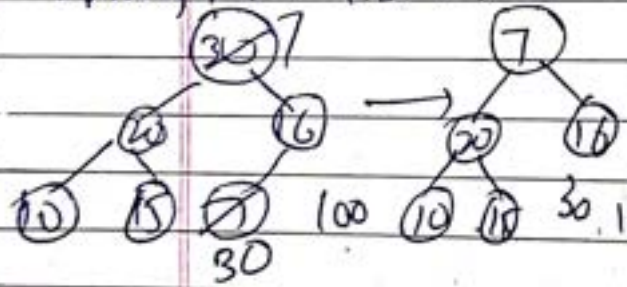
Scanned with CamScanner

HeapSort(A) {
BUILD MAX HEAP ()
for (i=A.length down to 2)
exchange A[1] with A[i]
A.heap_size = A.heap_size - 1
MAX HEAPIFY (A, 1)
}

eg



Apply MAX-HEAPIFY



and so on

Time complex $O(n \log n)$

In a heap with n elements with smallest element at root, the 7th smallest element can be found in time
a) $\Theta(n \log n)$ b) $\Theta(n)$ c) $O(\log n)$ d) $O(1)$

→ It is min heap (ATQ).
Delete 1st min $= O(\log n)$
" 2nd " $= O(\log n)$
" 3rd " $= O(\log n)$
" 4th " $= O(\log n)$
7th $= O(\log n)$

Scanned with CamScanner

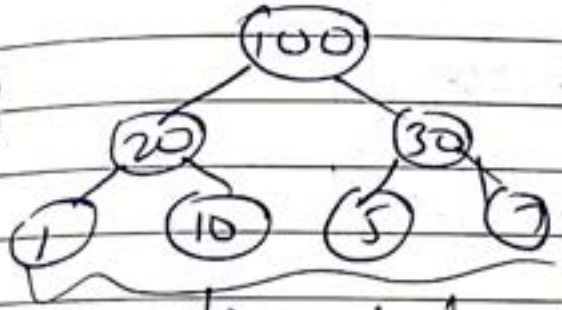
Find 7th - O(1)

Now insert all 6 back = 6 * O(log n)
= O(log n) (C) = 6 O(log n)

Q-1 In a binary max heap containing n elements the smallest element can be found in time

a) O(n) b) O(log n) c) O(log log n) d) O(1)

Ans



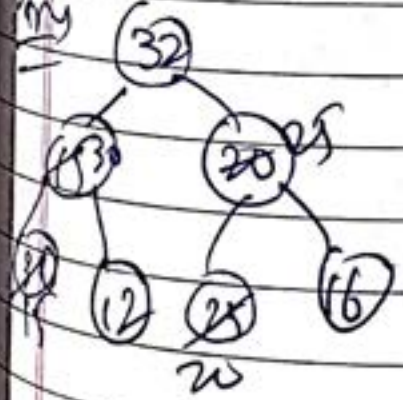
smallest element in leaves only

So go from $\lfloor \frac{n}{2} \rfloor + 1$ to n

Roughly $O(\frac{n}{2}) = O(n)$ a)

Q-35, 15, 20, 30, 12, 25, 16 inserted into max heap. What does heap look like

Ans

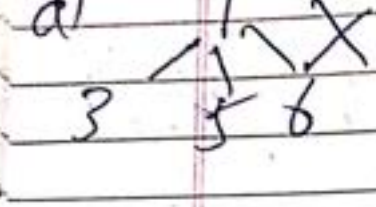


32 30 20 25 15 12 20 16

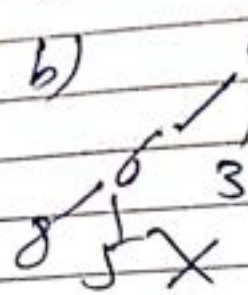
Scanned with CamScanner

→ Ternary max heap is represented by
a) 1, 3, 5, 6, 8, 9 (b) 9, 7, 5, 1, 8, 5 (c) 9, 5, 6, 8, 3, 1
d) 9, 5, 6, 8, 3, 1 & insert 7, 2, 10, 4

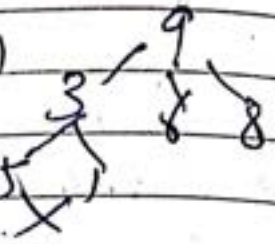
a)



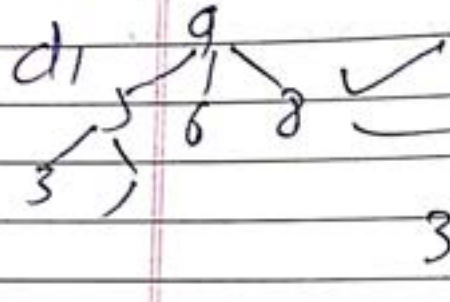
b)



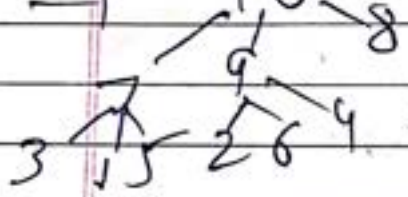
c)



d)



→ 10 7 9 8 3 5 2 6 4



Consider the process of inserting an element into a max heap. If we perform a binary search on the path from new leaf to root to find the position of newly inserted element, the # of comparisons performed are-

→ O(log n) (Assume any eg)
↓ time for binary search
total height

P.S - We can insert in O(log n) only.

Same ques is also asked in insertion sort



Insert O(n)
Binary O(log n)

Q. We have a binary heap on 'n' elements and wish to insert 'n' more elements (not necessarily one after another) into this heap. The total time required for this is
a) O(log n) b) O(n) c) O(n log n) d) O(n²)

Ans O(n) : we call build heap by combining all 2n elements if it would have been one after another then n x log n = O(n log n)

```
InsertionSort(A){
for j=2 to A.length
  key = A[j];
//insert A[j] into sorted sequence A[1..j-1]
i=j-1
while (i>0 && A[i]>key)
    A[i+1] = A[i]
    i = i-1;
A[i+1] = key
```

Insert 9 6 5 0 8 2 7 3

9 → 69 → 659 → 569 → 5690 → 5069 → 0569

0569 → 05689 → 056892 and so on —

Scanned with CamScanner

Worst Case — comparison ↑ movement

for $5 = 2 - 1 + 1 = 2 = 2(1)$
$3 - 2 + 2 = 4 = 2(2)$
$4 - 3 + 3 = 6 = 2(3)$
$5 - 4 + 4 = 8 = 2(4)$

$n_1 \longrightarrow 2(n-1)$

$O(n)$ So $2(1+2+\dots+n-1)$
$= O(n^2)$

Best Case $O(n)$

Space complexity $O(1)$
i.e inplace algorithm

In insertion search even if we replace comparison by binary search $O(\log n)$ instead of linear search $O(n)$ then
$O(n) \quad O(n)$

$O(n + \log n) = O(n)$

If we use Doubly LL also movement can be $O(1)$ but still time complexity $= O(n)$

$\therefore O(n+1) = O(n)$

Imp: We can't apply binary search on Doubly LL

Better than insertion sort

Merge sort

Date | Page | 243

Let's talk about merging first

1	2	3	q	q+1	6	7	r
1	5	7	8	2	4	6	9

Now 1 to 4 are sorted and 5 to 8 are sorted, we want to merge these 2 but

Whenever an array is given such that p to q is sorted & q+1 to r is sorted, we create 2 arrays L & R

L				R			
1	5	7	8	2	4	6	9
i				j			

We are putting ∞ at end of L & R

1	2	3	4	5	6	7	8
1	2	4	5	6	7	8	9
k							

compare i and j whichever lesser put in array & increment i or j whichever

```
MERGE (A, p, q, r){
n1 = q - p + 1
n2 = r - q
let L[1 ... n1+1] & R[1 to n2+1] be new arrays
for (i = 1 to n1)
    L[i] = A[p + i - 1]
for (j = 1 to n2)
    R[j] = A[q + j]
L[n1+1] = ∞
R[n2+1] = ∞
i = 1, j = 1; for (k = p to r) if (L[i] <= R[j])
A[k] = L[i]; i++
else A[k] = R[j]; j++
```



Scanned with CamScanner

Space complexity $\Theta(n+m)$

Why ∞ added at end?

Let's see by eg

p q q+1 r

A [10|20|30|40] [1|5|6|9]

[10|20|30|40|∞] [1|5|6|9|∞]

[1|5|6|9] ← So at last compared with ∞

So benefit of ∞ is that we need not to worry about n & m

Now let's see Merge Sort (It falls in DAC (category) Divide & Conquer)

Merge-Sort (A, p, r) {
if p < r
q = ⌊(p+r)/2⌋
1 merge-sort (A, p, q)
2 merg-sort (A, q+1, r)
3 merge (A, p, q, r)
}

P-S - Merge & merge sort on left.

[9|6|5|0|8|2]
1 2 3 4 5 6

MS(1,6) ⌊(1+6)/2⌋ = 3

MS(1,3) MS(4,6) M(1,3,6)

MS(1,2) MS(3,3) M(1,2,3) MS(4,5) MS(6,6) M(4,5,6)

M(1,1,2) MS(4,4) MS(5,5) M(4,5,5)

Scanned with CamScanner

Merge → Always 2 sorted lists
Merge sort → Any 2 lists (sorted or unsorted)

Space complexity → height of tree
O(logn)
merge → O(n)
Stack → O(logn)
Total → O(n+logn) = O(n)

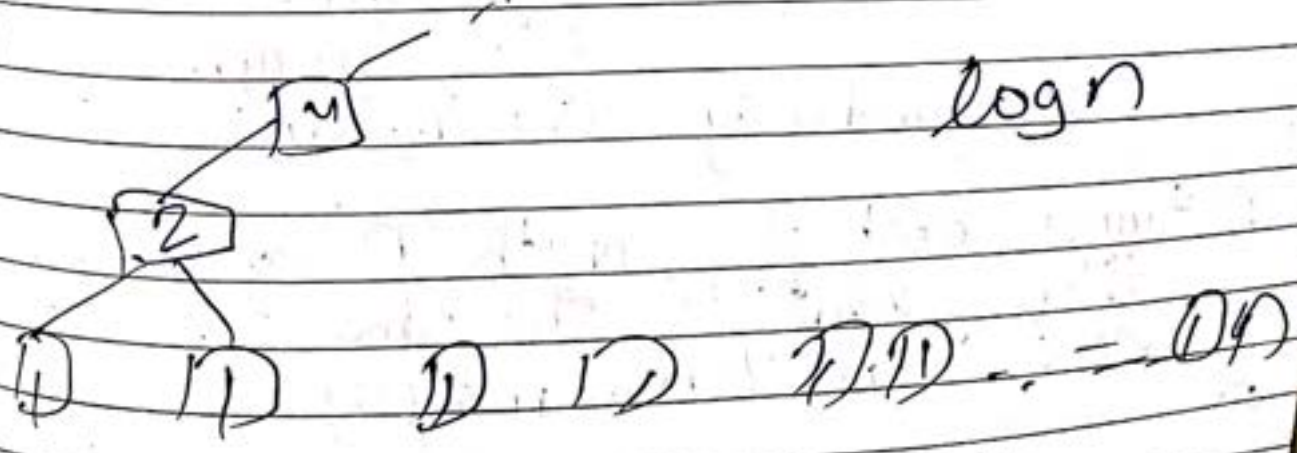
~~out of place~~ algorithm

Time complexity →

1- T(n/2)
2- T(n/2)
3- O(n)

So T(n) = 2xT(1/2)+O(n)
Master theorem
= θ(nlogn)

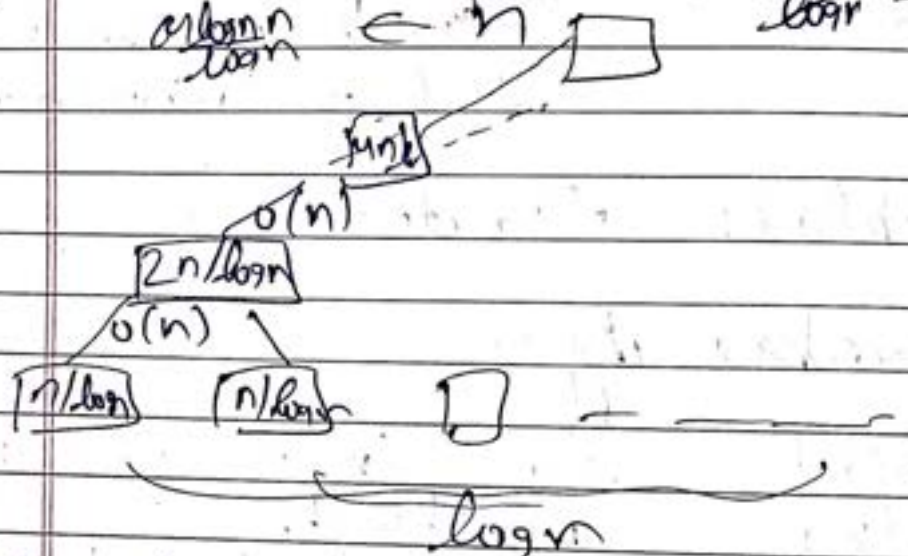
Given n elements, merge them into one sorted list using merge function



Scanned with CamScanner

Q. Given "logn" sorted lists each of size "n/logn", what is the total time required to merge them into one single list -

(A) → Total element = $\log n \times \frac{n}{\log n} = n$



Total climbing = n

Total time for 1st climbing log 2
2nd log 4
3rd log 8
nth loglogn

So Time Complexity O(nloglogn)

n strings each of length 'n' are given then what is the time taken to sort them 2 way merging.

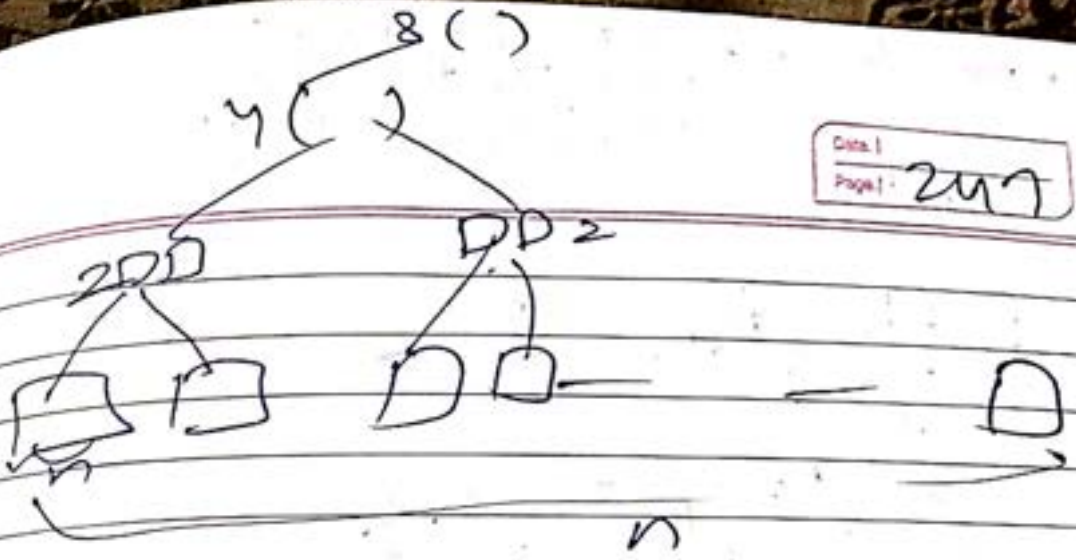
Ans

a ab aaa

so O(n)

2 strings

n n n



At first level comparisions = $O(n/2)$

for every comparison time taken = $O(n)$

So time $O(n) \times O(n) = O(n^2)$

Now for copy $O(n^2)$ So $O(O(n^2) + O(n^2)) = O(n^2)$

At every level $O(n^2)$ work

Total climbing $\log_2 n$

So $O(n^2 \log_2 n)$

1) Merge sort uses
a) DAC b) Greedy c) Backtracking

2) 'm' & 'n' time taken
$O(m+n)$

3) find after 2 passes . 20, 47, 15, 8, 9, 4, 40, 30, 12, 17

20 47 15 8 9 4 40 30 12 17

(20,47) (8,15) (4,9) (30,40) (12,17)

(8,15,20,47) (4,9,30,40) (12,17)

Scanned with CamScanner

Both Merge & Quick sort follow DAC.
Merge sort → Merge
Quick sort → Partition

Quick Sort

```
Partition(A,p,q){
    x = A[q]
    i = p-1
    for(j = p to q-1){
        if(A[j] ≤ x){
            i = i+1
            exchange A[i] with A[j]
        }
    }
    exchange A[i+1] with A[q]
    return i+1
}
```

Above done assuming pivot at end
→ You know pivot at middle is bad choice

Time complexity $O(n)$

i j
Element sorted at 1 partition

13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11
1 2 3 4 5 6 7 8 9 10 11 12

If j<n incremented & swapped with i & i incr
else j>11 j incremented

i j j j j
13 19 9 5 12 8 7 4 21 ... and so on
5 8 19 12 13

Date. / Page. / 249

Quicksort (A, p, r) {
if (p < r) {
q = Partition (A, p, r)
1. Quicksort (A, p, q-1) } DAC
2. Quicksort (A, q+1, r)
}
}

QS(1,7)

P(1,7) (4) index QS(1,3) QS(5,7)

P(1,3) (2) QS(1,1) QS(3,3) P(5,7) (5) QS(5,4) QS(6,6)

	1	2	3	4	5	6	7
A	81	73	62	84	37	26	45

P(1,7)

A	1	32	23

P(1,3)

75	6	87

P(5,7)

Stack Height

Space O(log n) if balanced

In worst case true unbalanced O(n)

n

1 n-2

1 n-4

space O(n)

Time complexity

1 → O(n)

2 → T(n/2)

3 → T(n/2)

T(n) = 2T(n/2) + O(n)

= O(n log n)

Worst Case, O(n²)

∴ T(n) = T(n-1) + O(n)

3 ↓ 2 → 1

Scanned with CamScanner

Let's give I/P to quicksort (1, 2, 3, 4, 5, 6)

(1 2 3 4 5 6)
i j n-1 (pivot)

B.C. exchanged nothing
Filling 2 & 2 is B.C.

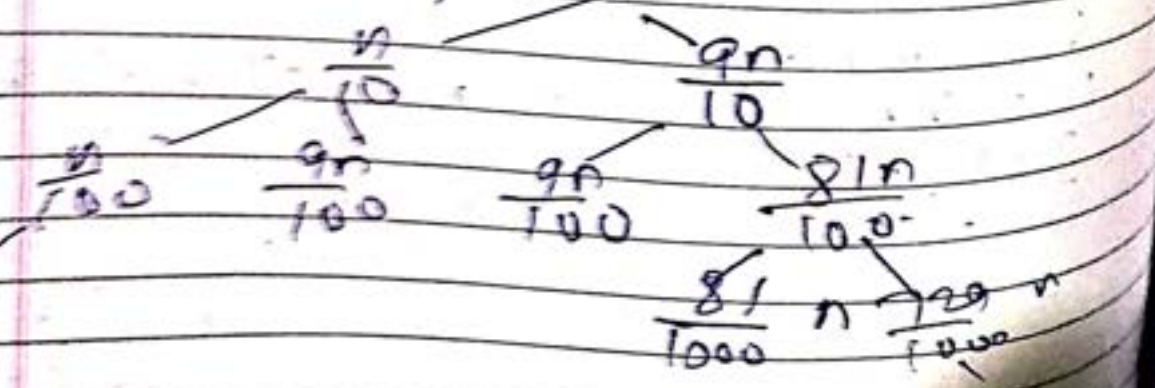
$T(n) = O(n) + T(n-1)$
$= O(n^2)$

Worst given (6 5 4 3 2 1)
$\rightarrow O(n^2)$

→ 2 2 2 2 2 2
i j ↑
$\rightarrow O(n^2)$

Best case $O(n \log n)$ worst $O(n^2)$

Division ratio 1:9



Scanned with CamScanner

Date:
Page: 251

Max levels

$n - \frac{n}{10/9} - \frac{n}{(10/9)^2} \quad \cdots : 1 : \log_{10/9} n$

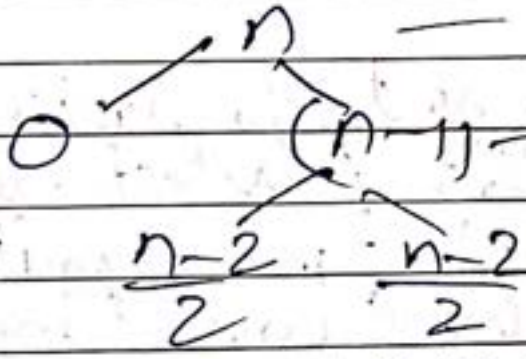
$= \Theta(\log_2 n)$

So $\Theta(n \log n)$

If ratio (split) is not as bad as $0 : (n-1)$

Even 1 : 9, 1 : 99, 1 : 999 have good time complexities

Fig 2



$T(n) = cn + cn + 2T\left(\frac{n-2}{2}\right)$

$= 2cn + 2T\left(\frac{n-2}{2}\right)$

$\leq O(n) + 2T\left(\frac{n}{2}\right)$

$\Theta(n \log n)$

So if $0 : n-1$ at every level then also $\Theta(n \log n)$

Scanned with CamScanner

Q) The median of 'n' elements can be found in O(n) time. which one of the following is correct about complexity of quick sort, in which median is selected as pivot?

Ans 3 steps
O(n) → Median
O(1) → Median at end
O(n) → Quick sort
$T(n) = O(n) + 2T(\frac{n}{2}) = O(n \log n)$

Q) In quick sort, for sorting 'n' elements the $(\frac{n}{4})^{th}$ smallest element is selected as pivot, using O(n) time algorithm. what is the worst case time complexity of quick sort?

Ans O(n) → n/4
O(1) → At end
O(n) → QuickSort Partition

n/4	3n/4

i.e 1 : 3 $O(n \log n)$
You can do by master theorem
also $T(n) = O(n) + T(\frac{n}{4}) + T(\frac{3n}{4})$

Scanned with CamScanner

Q: 1, 2, 3, 4, ——, n —— T_1 (Time complexity)
n, n-1, n-2 —— —— T_2
Relation b/w T_1, T_2

Ans
$T_1 = T_2$

Q $O(n)$ ≤ $\frac{1}{5}n$, $\frac{4}{5}n$ time complexity

$T(n) = O(n) + T(\frac{n}{5}) + T(\frac{4n}{5})$

$T(n) \leq O(n) + 2T(\frac{4n}{5})$

Radix_sort (A, d)

Use any stable sort like counting sort

804		804		001
026	Count	026	Apply	052
005	all	005	Any	804 → unit place
64	to 3 digit	064	sort	064
52		052	on	005
1		001	unit	026
			digit	

Ten's digit

001		001
804	Hundredth	005
005	Digit	026
026		052
052		064
064		804

Scanned with CamScanner

RadixSort(A, d) {
// each key in A[1..n] is a d-digit
(digits) are numbered 1 to d from R→L

for i = 1 to d :
Use a stable sorting algorithm to sort A on digit i
↓
maintaining order of other keys

Counting sort prefered : in decimal 0-9 range
binary 0-1 range
octal 0-7 ..
etc.

Time Complexity $\log_b(O(n+b))$

b - base (10 in decimal, 2 in binary)
l - largest number
$O(n+b)$: Counting sort

Assume l very big (say n^k)

$\log_b n^k (O(n+b))$

$k \log_b n (O(n+b)) = O(n \log_b n)$
Counting sort

Space comp $O(b)$

Best case $O(n)$

Scanned with CamScanner

Crack 2016

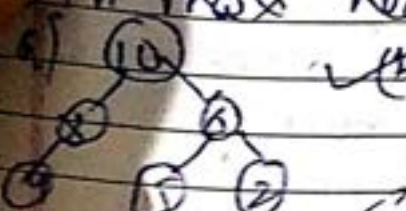
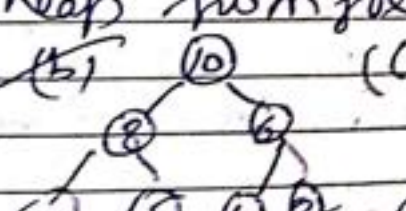
Q1 The worst case running times of Insertion sort, merge sort & quick sort, respectively are:

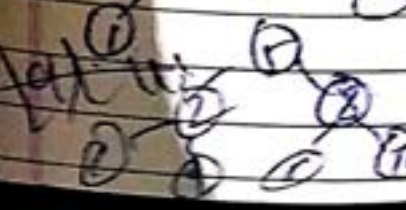
a) $\Theta(n \log n)$, $\Theta(n \log n)$ & $\Theta(n^2)$
b) $\Theta(n^2)$, $\Theta(n^2)$ & $\Theta(n \log n)$
c) $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$
d) $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

Algo	Best Time	Avg Time	Worst Time	Worst Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary "	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection "	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion "	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge "	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick "	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap "	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Bucket "	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Radix "	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$
Tim "	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Shell "	$O(n)$	$O(n \log n)^2$	$O(n \log n)^2$	$O(1)$

2011

A max heap from fold is

a) 
b) 
c) 

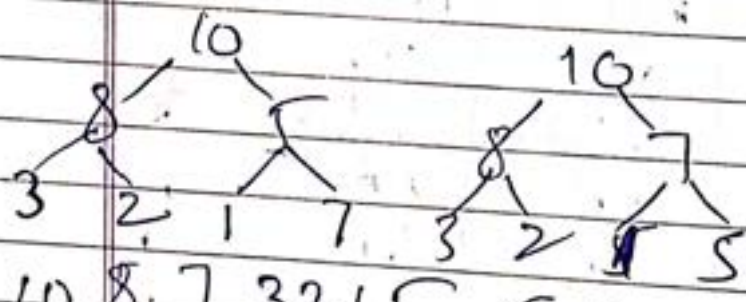


Scanned with CamScanner

Gate-2014, 2005

Q: A priority queue is implemented as a max-heap initially it has 5 elements. The level order traversal of the heap is: 10, 8, 5, 3, 2. Three new elements (8, 7, 10) are inserted into heap in that order. The level order traversal of heap after insertion is

a) 10, 8, 7, 3, 2, 8, 5
b) 10, 8, 7, 2, 3, 8, 5
c) 10, 8, 7, 1, 2, 8, 5
d) 10, 8, 7, 5, 3, 2, 1



10 8 7 3 2 8 5 (a)

Gate 2016

Q: An operator delete(i) for a binary heap DS is to be designed to delete the item in the ith node. Assume that the heap is implemented in an array and i refers to the ith index of the array. If the heap tree has depth 'd' (number of edges on the path from root to the farthest leaf), then what is the time complexity to re-fix the heap efficiently after the removal of the element

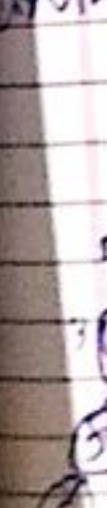
a) O(1) (b) O(d) but not O(1)
(c) O(2^d) but not O(d)
(a) O(d 2^d) not O(2^d)

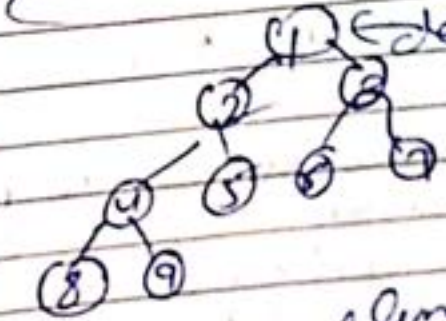
Scanned with CamScanner

Lab 2010

Q A complete binary min-heap is made by including each integer in [1, 1023] exactly once. The depth of a node in the heap is the length of the path from the root of the heap to that node. Thus the root is at depth 0. The max depth at which integer 9 can appear is—

Ans: Let's analyze





depth = 8

depth = 3

d = 0 element 1
d = 1 2
d = 2 4
d = 8 $2^8 = 256$

$256 + 128 + 64 + 32 + 16 + 8 + 4 + 2$
8 7 6 5 4 3

→ 512

so answer yes

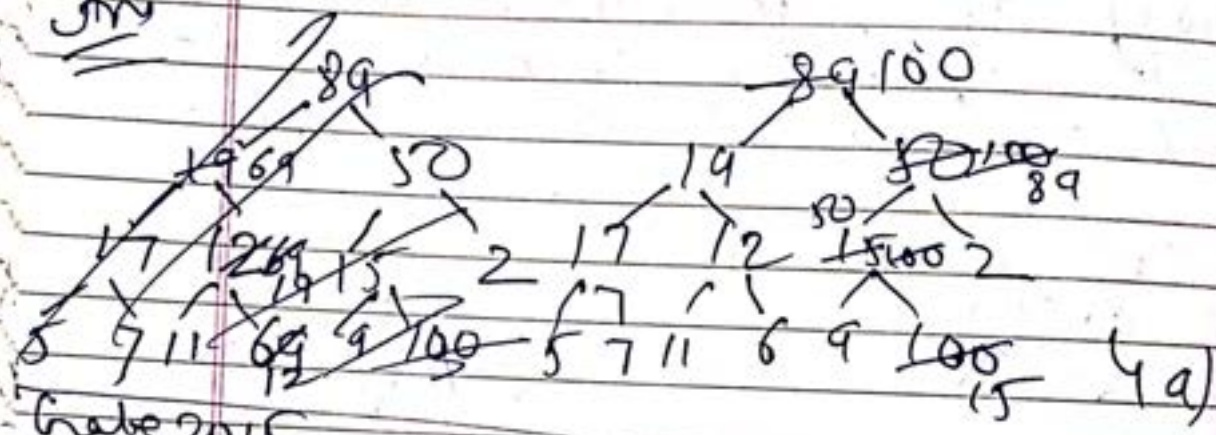
8 is ans

Gate 2015

Q Consider the following array of elements <89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100>. The min # of interchanges needed to convert into a maxheap is

a) 4 b) 5 c) 2 d) 3

Ans



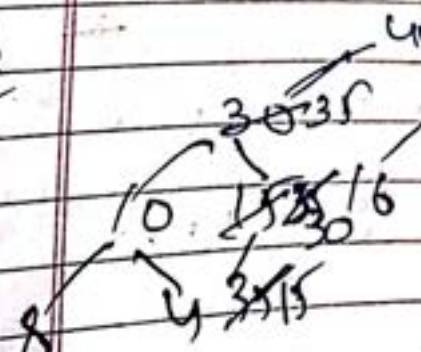
4 a)

Gate 2015

Q Consider a maxheap represented by the array 40, 30, 20, 10, 15, 16, 17, 8, 4. Now consider that a value 35 is inserted into this heap. After insertion the new heap is

a) 40, 30, 20, 10, 15, 16, 17, 8, 4, 35
b) 40, 35, 20, 10, 30, 16, 17, 8, 4, 15
c) 40, 30, 20, 10, 35, 16, 17, 8, 4, 15
d) 40, 35, 20, 10, 15, 16, 17, 8, 4, 30

Ans



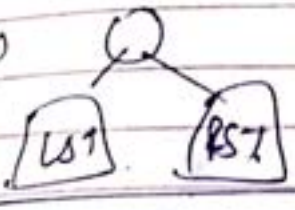
40, 35, 20, 10, 30, 16, 17, 8, 4, 15 (b)

Scanned with CamScanner

June 2015

Consider a complete binary tree where the left & right subtree of the root are max-heaps. The lower bound for # of operations to convert tree to a heap is:
a) $\Omega(\log n)$ b) $\Omega(n)$ c) $\Omega(n \log n)$ d) $\Omega(n^2)$

Ans



Binary Search

→ Also know as half-interval search or logarithmic search, is a search algorithm that finds the position of a target value within a sorted array. It does not work on unsorted array

How it work?

⇒ Binary search works on sorted arrays. It begins by comparing the middle element of the array with target value
→ If target value matches the middle element, its position in the array is returned
→ If the target value is less than the middle element then search continues in left half of the array otherwise search continues in the right half of the array

Problem Statement

→ Given an array A of n elements with values or records $A_0 \ldots A_{n-1}$ sorted such that $A_0 \leq \ldots \leq A_{n-1}$ and the target value T, the following subroutine uses binary search to find index of T in A.

Algo

1) Set L to 0 & R to n-1
2) If L > R, the search terminates as unsuccessful.
3) Set m (the position of middle element) to the floor (the largest previous integer) of (L+R)/2
4) If T = Am the search is done; return m.
5) If T > Am, set L to m+1 & go to step 2
6) If T < Am, set R to m-1 & go to step 2

Implementation in array

```
int binarySearch(int arr[], int l_index,
    int r_index, int target) {
int m_index;
while (l_index <= r_index) {
    m_index = l_index + (r_index - l_index)/2;
if (arr[m_index] == target)
    return m_index;
if (arr[m_index] < target)
    l_index = m_index + 1;
else
    r_index = m_index - 1;
}
return -1;
}
```

Scanned with CamScanner

Binary search DAC

Date:
Page: 261

Time Complexity:

Recurrence relation: $T(n) = T(n/2) + 1$
$= 1$ if $n = 1$

$T(n) = O(\log_2 n)$

Searching algorithms -

→ In C.S, search algo is an algo that retrieves information stored within some data structures.
→ Data structure can include Linked lists, arrays, search trees, hash tables, or various other storage methods.
→ Searching also encompasses algorithms that query the database such as the SQL SELECT command.

Linear search (or sequential search) Method for finding a target value within a list

How it works!
→ It sequentially checks each element of the list for the target value until a match is found or until all the elements have been searched.
→ It works both on sorted & unsorted data.

Scanned with CamScanner

Problem Statement: Given a list of n elements with values or records L[0] ... L[n-1], & target value T, Fall subroutine uses linear search to find the index of the target T in L

Basic Algorithm:

1) Set i to 0.
2) If L[i] = T, search terminates, return i.
3) Increase i by 1.
4) If i < n, go to step 2. otherwise, the search terminates unsuccessfully.

Implementation of linear search

```
int linearSearch(int arr, int size,
                 int target)
{
    for(int index=0; index<size; index++)
        if(arr[index] == target)
            return index;
    return -1;
}
```

Time Complexity

Approach 1

Best Case = 1
Worst Case = n
Avg Case = n

Approach 2

T(n) = T(n-1) + 1 if n > 1
T(1) = 1
T(n) = O(n)

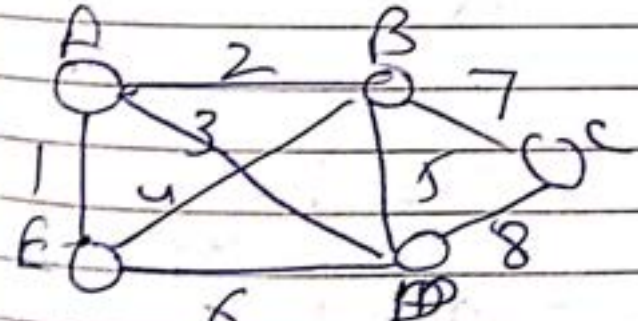
Scanned with CamScanner

Implementation using LL


```
struct node *linearSearch_LL(struct node *head, int target){
    if(head){
        while(head){
            if(head->data == target)
                return head;
            head = head->next;
        }
    }
    return head;
}
```

Intro to Greedy Algos

optimisation problems
→ min cost
→ max profit
→ max reliability
→ min risk



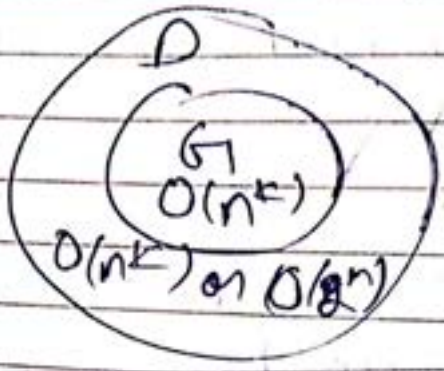
G → shortest path from A to C.

1 way → find all possible values & then take least. $O(2^n)$
2 more ways → Programming Paradigms

Greedy → (not all prob solved) $O(n^k)$
DP → $O(2^n)$ (almost all problems)

Scanned with CamScanner

All optimisation Problems



Knapsack Problem

m = 20, get max profit → Greedy about profit

	ob1	ob2	ob3
Profit	25	24	15
Weight	18	15	10

So 18 → ob1, we find percent

ob1 = $\frac{25}{18}$ = 1.41

ob2 = $\frac{24}{15}$ = 1.6

ob3 = $\frac{15}{10}$ = 1.5

ob2 → 15

ob3 → 5

profit = 24 + 18 × 5

= 31.5

Scanned with CamScanner

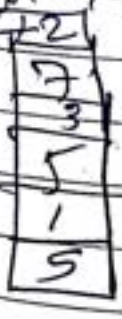
Date / Page 265

Algorithm
Greedy knapsack(
for i=1 to n;
compute pi(wi); // O(n)
sort objects in non increasing order of p/w
for i=1 to n from sorted list } O(nlogn)
if (m>0 && wi ≤ m) } O(n)
m = m - wi;
p = p + pi;
else break; // out of for loop
if (m>0)
p = p + pi(m/wi)
}

eg n=15 n=7

objects	1	2	3	4	5	6	7
profits	10	5	15	7	6	18	3
weight	2	3	5	7	1	4	1
	5	1.6	3	1	6	4.5	3

Obj# sorted
selected 5, 1, 6, 3, 7, 2, 4

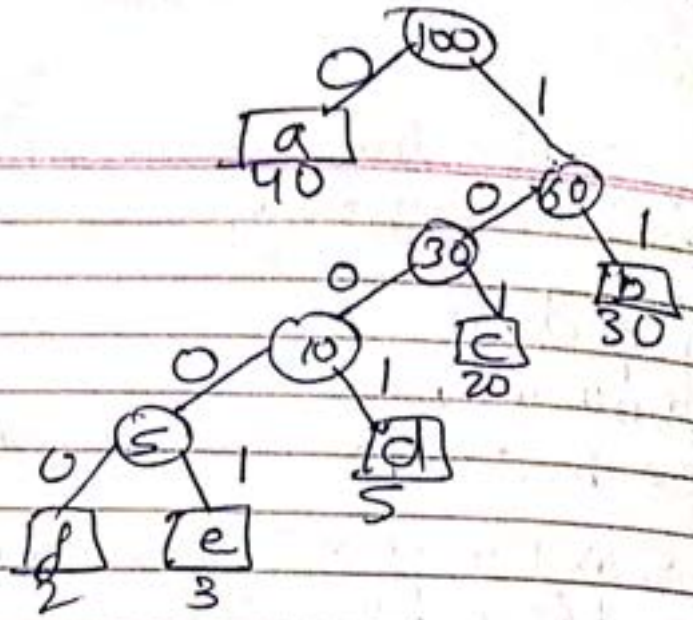


m = 15 14 12 8 5 2 0
p = 6 + 10 + 4.8 + 15 + 3 + 5 + 2/3
= 55.3

Time Complexity O(nlogn) see labels in code
Due to insertion by max heap
O(n) + n O(logn) = O(nlogn)
sort delete
so no benefit

Eg

a-40
b-30
c-20
d-5
e-3
f-2
100



$40\times1+30\times2+20\times3+4\times5+2\times5+3\times5$
$=205$

$\frac{205}{100}=2.05$

a = 0
b = 11
c = 101
d = 1001
e = 10001
f = 10000

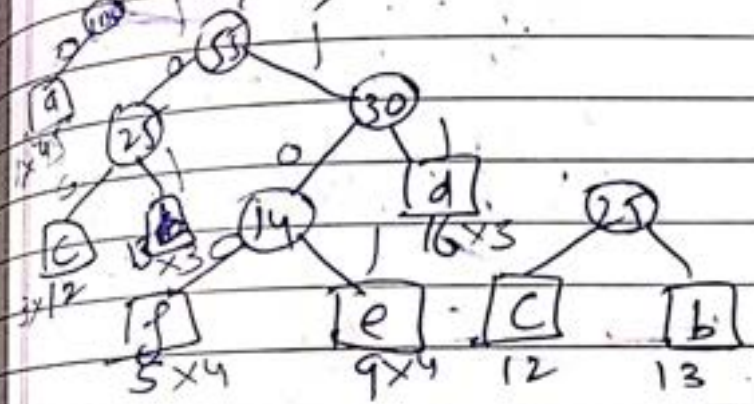
Prefix code i.e. if 0 is prefix of a it's only of a
11 - of b
101 of c
1001 of d etc.

```
HUFFMAN(C){
n=|C|
make a min heap 'Q' with C
for i=1 to n-1
    Create a new node z
    z.left = x = Extract-min(Q)
    z.right = y = Extract-min(Q)
    z.freq = x.freq + y.freq
    Insert(Q,z)
return(Extract-min(Q))
```

4 character

a	b	c	d	e	f
45	13	12	16	9	5

Q. 45, 13, 12, 16, 9, 5 → 45, 13, 12, 16, 14
→ 45, 25, 16, 14 → 45, 25, 30 → 45, 55



a - 0
b - 101
c - 100
d - 111
e - 1101
f - 1100

Time Complexity

for loop - O(nlogn)
Make min heap O(logn)
Totatal - O(nlogn)

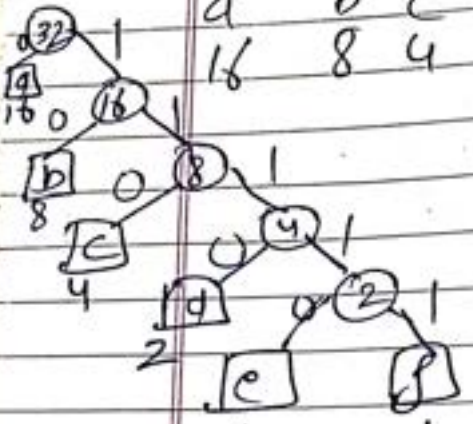
Space complexity - O(n)

Q.
Suppose, the letters a, b, c, d, e, f have probabilities 1/2, 1/4, 1/8, 1/16, 1/32, 1/32
i) Which of the following is the huffman code for the letter a, b, c, d, e, f?
a) 0, 10, 110, 1110, 11110, 11111
b) 11, 10, 011, 010, 001, 000
c) 11, 10, 01, 001, 0001, 0000
d) 110, 100, 010, 000, 001, 111

Scanned with CamScanner

Ans
Idon't want fractions, so I multi by 32

a	b	c	d	e	f
16	8	4	2	1	1



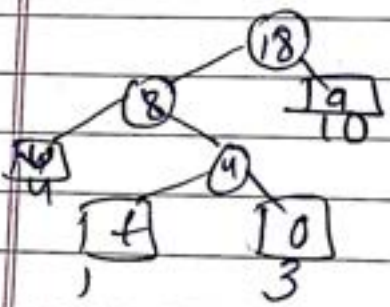
(a) ✓

(b) what is average length of the longest answer to above a
a) 3 b) 2.1875 c) 2.25 d) 1.9375

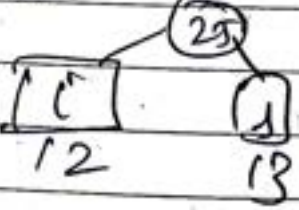
$$\frac{16\times1+8\times2+4\times3+2\times4+1\times5}{16+8+4+2+1+1}=\frac{63}{32}=1.937$$

(c)

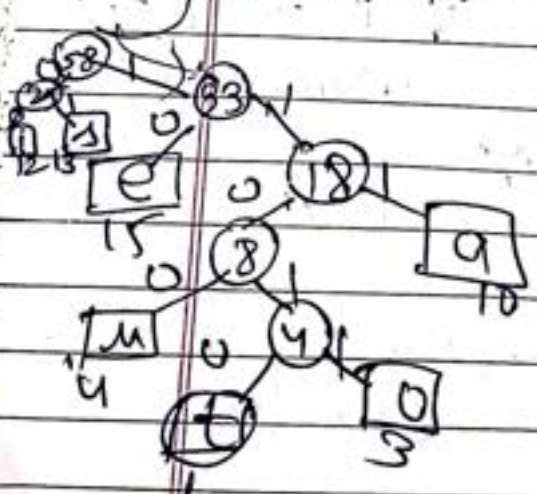
a	e	i	o	u	s	t
10	15	12	3	4	13	1



→ Now 12 & 13 are smaller than 18 so we build a separate tree



18, 15, 25 left



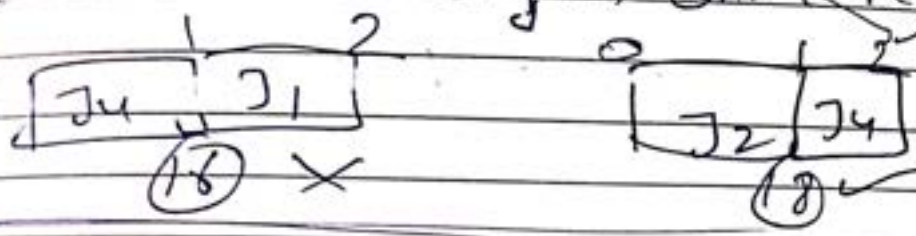
a = 111
e = 10
c = 00
o = 11011
u = 1100
s = 01
t = 11 or 0

Job sequencing with deadline

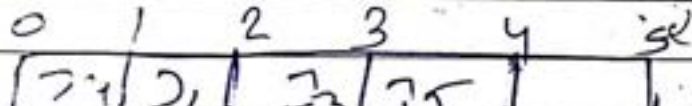
	J_1	J_2	J_3	J_4
D	2	1	2	1
P	6	8	5	10

D-Deadline
P- Profit only when job done within deadline

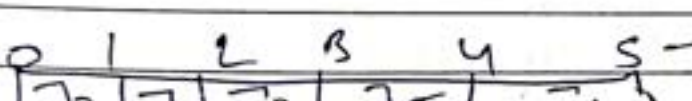
Every Job will take 1 unit of time to complete & jobs can't be done simultaneously



	J_1	J_2	J_3	J_4	J_5	J_6
D	5	3	3	2	4	2
P	200	180	190	300	120	100



= 810 (Decreasing order of profit)



= Highest profit as far as possible

= 990

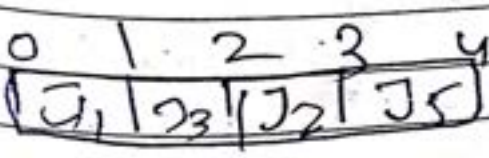
Worst case $O(n^2)$ — $O(n \log n)$ for sorting

n jobs n searches.

Total $O(n^2 + n \log n) = O(n^2)$

Jobs	J_1	J_2	J_3	J_4	J_5
Profits	2	4	3	1	10
Deadlines	3	3	3	4	4

Sort J_5, J_2, J_3, J_1, J_4



Ans = 19

Scanned with CamScanner

	J_1	J_2	J_3	J_4	J_5	J_6	J_7	J_8	J_9
P	15	20	30	18	18	10	23	16	25
D	7	2	5	3	4	5	2	7	3

7 → max deadline available

0	1	2	3	4	5	6	7
	J_2	J_7	J_9	J_1	J_6	J_7	J_8

Sort J_3 J_9 J_7 J_2 ~~J_4~~ J_5 J_8 J_1 ~~J_6~~

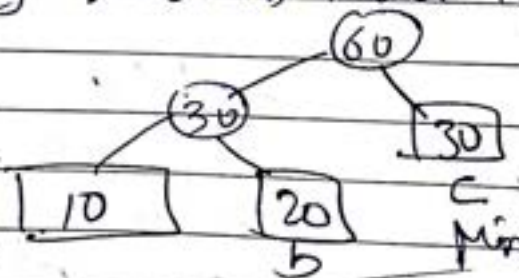
$30+25+23+20+18+16+15$

=

OPTIMAL Merge Pattern

files a (10 records), b (20), c (30)

Total # of record movements must be as min as possible



Min weighted path length

1 ↳ Huffman coding

Ans = $2\times10+2\times20+1\times30$ ~~$+1\times30$~~

= 90

Easy Way 10 record copied from a → 30

20 — — — b → 30

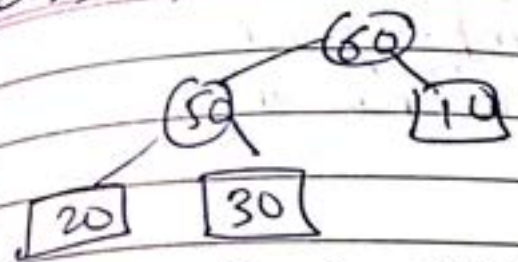
Total = 20 + 10 = 30

Now 30 copied from 30 — c → 60 Total 30+30 = 60

30 " " " c → 30 — 60+30 = 90

Scanned with CamScanner

Let's see another combination

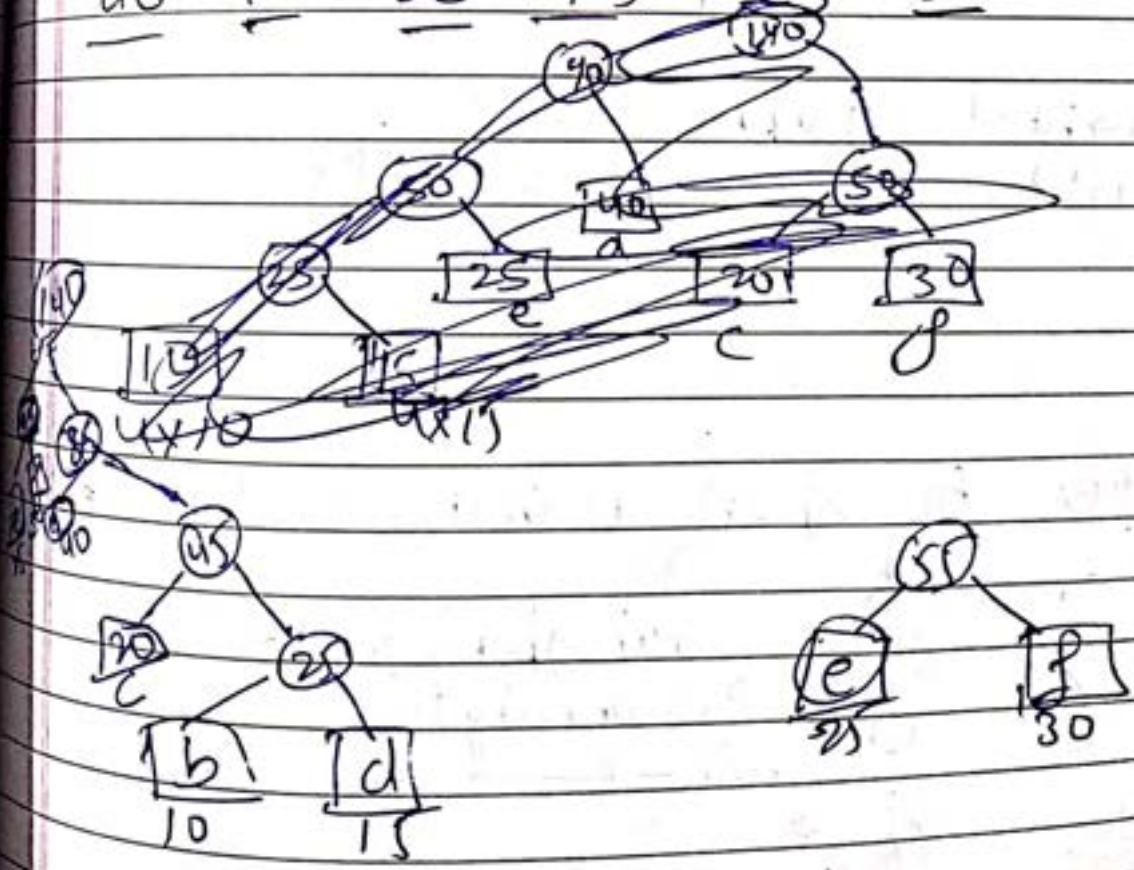


$2 \times 20 + 2 \times 30 + 1 \times 10 = 110$ X

so huffman is minimal

O(nlogn) → Huffman

a	b	c	d	e	f
40	10	20	15	25	30



Ans = $4 \times 15 + 4 \times 10 + 3 \times 20 + 2 \times 40 + 2 \times 25 + 2 \times 30$

$= 320$

Scanned with CamScanner

Graphs → no 2 edges b/w any vertices and no self loop
Simple graph
multiple graph

(V, E)

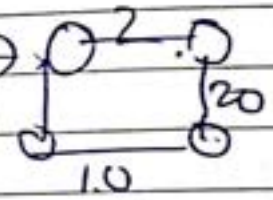
null graph

min edges in simple graph = 0
max edges = $^nC_2$

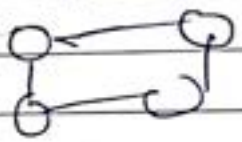
eg 

$\frac{n(n-1)}{2} = 6$

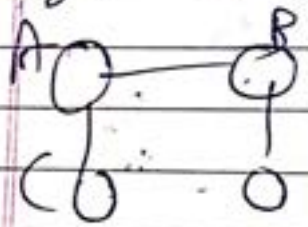
$E = O(V^2)$ worst case

Sp Weighted graph → 

UnWeighted

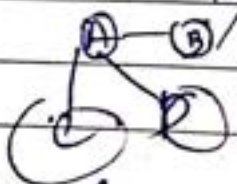


Min edges to span n vertices (spanning tree)



Min span edges = 3

→ Spanning tree



3 here

No. of edges of spanning tree possible = n-1
for n vertices

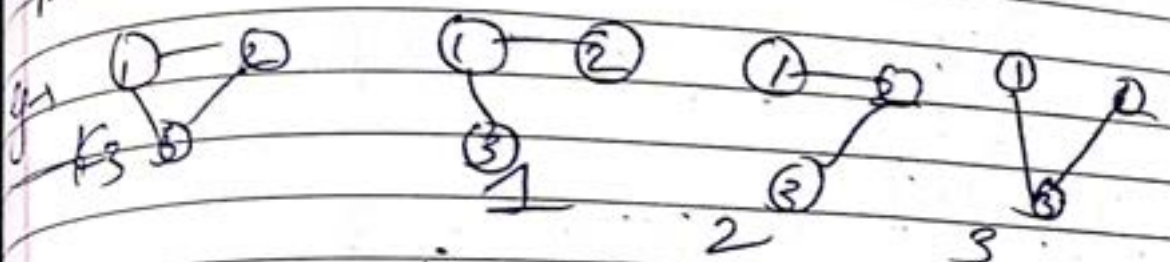
(Unweighted)

Scanned with CamScanner

Complete graph → In which every vertex connected with all other vertices

Date
Page 215

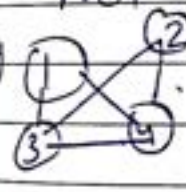
How many such spanning trees are present for 4 vertices.



3 for labeled, 1 for unlabeled.

3 — 3 (K_3),
4 — 16 (K_4) formula $K_n - n^{n-2}$

of spanning trees for graph which is not complete



Kirchoff's theorem
Draw matrix (adjacency)

	1	2	3	4
1	0	0	1	1
2	0	0	1	1
3	1	1	0	1
4	1	1	1	0

Connected graph → there is diagonal contains 0 and in adjacency matrix diagonal contains degree

(i) Diagonal 0's → degree
(ii) non diagonal 1's → 0's
(iii) Non diagonal 0's → 0's

	1	2	3	4
1	2	0	−1	−1
2	0	2	−1	−1
3	−1	−1	3	−1
4	−1	−1	−1	3

of spanning trees = find cofactor of any element

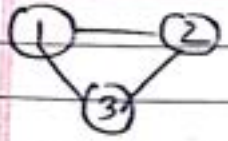
$\therefore e_{11} \times \begin{vmatrix} 2 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 3 \end{vmatrix}$

$= 2(9-1) - (-1)(-3-1) + (-1)(1+3)$

$= 8$ (complete K_4 it's 16)

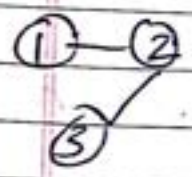
Scanned with CamScanner

K_3



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \end{array}$$

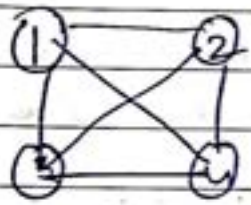
$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix} \end{array} = (2 \times 2 - 1) = 3$$



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \end{array}\begin{array}{c} \begin{array}{ccc} 1 & 2 & 3 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \end{array} \Rightarrow \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix}$$

$$2(x1) - 1 = 1$$

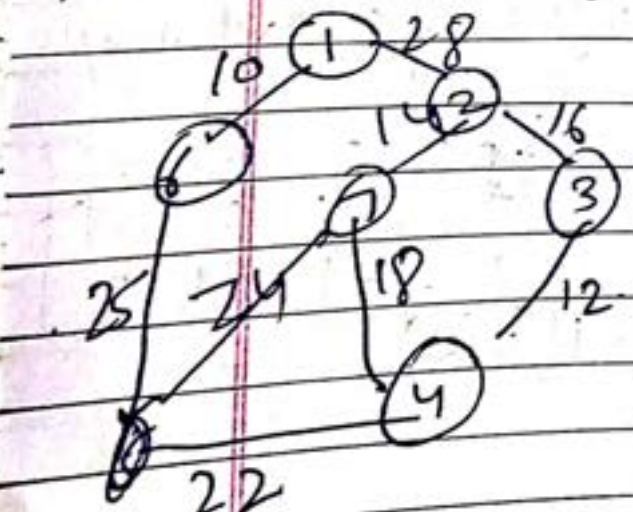
K_4



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}$$

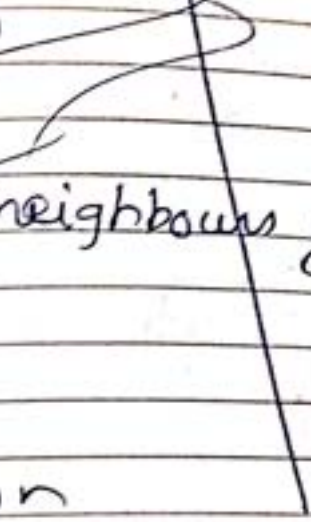
$$= 3(9-1) - (-1)(-3-1) + (-1)((+3)$$
$$= 24 - 8 = 16$$

Now weighted graph → Both are greedy (Kruskal then) Prim's Algo
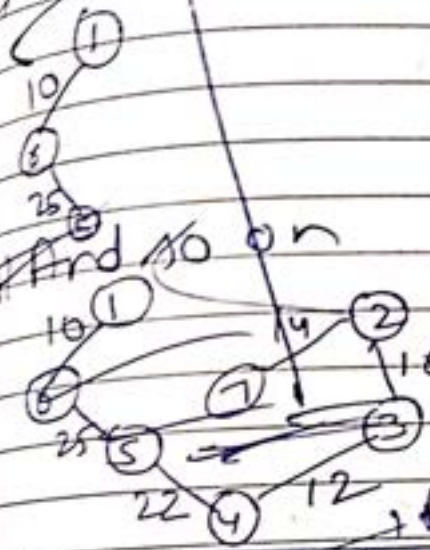


Scanned with CamScanner

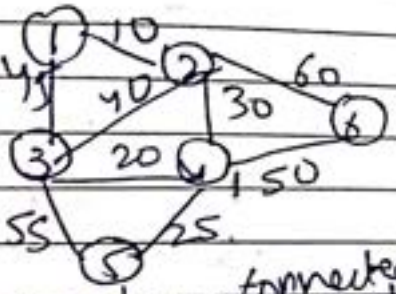
1) Take min weighted edge & put it in MST



2) Then the neighbours of 6 & 1

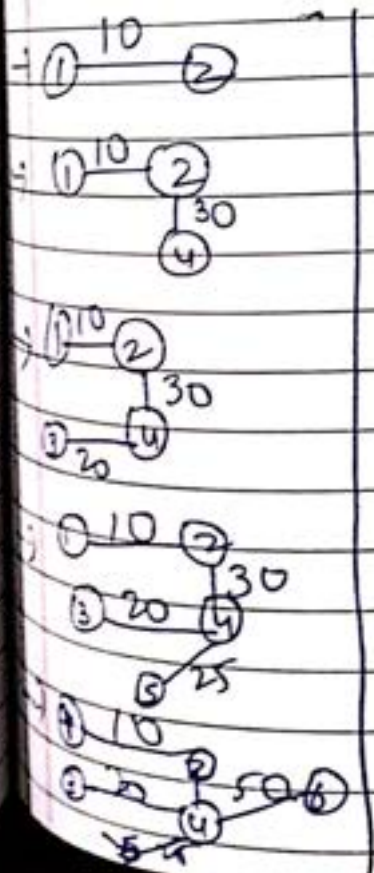


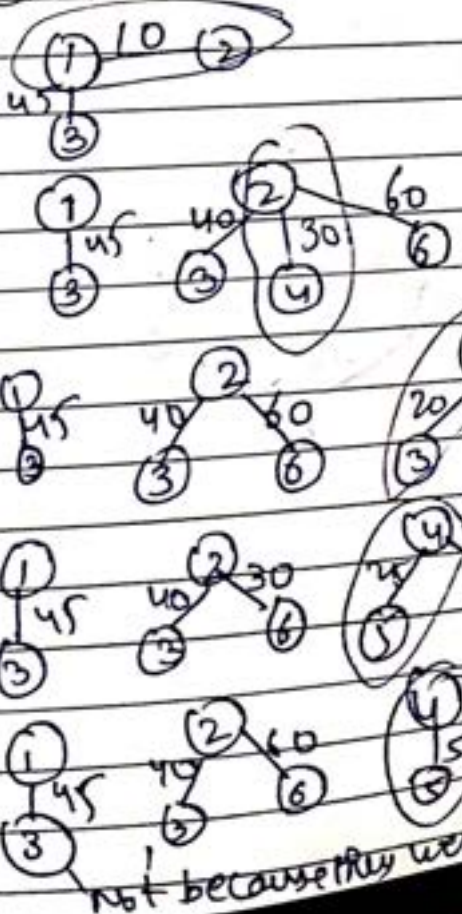
Ard so on



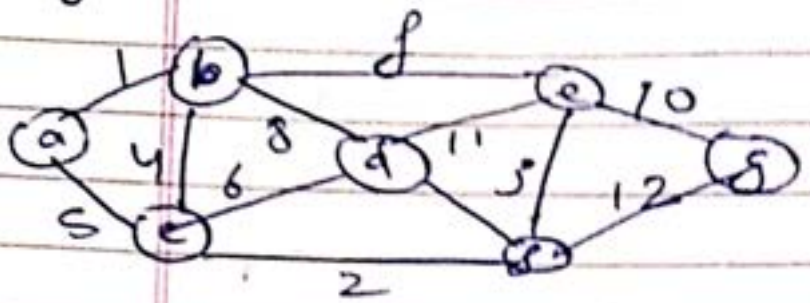
Always creates a connected graph.

Prim's Algo can be started from any vertex

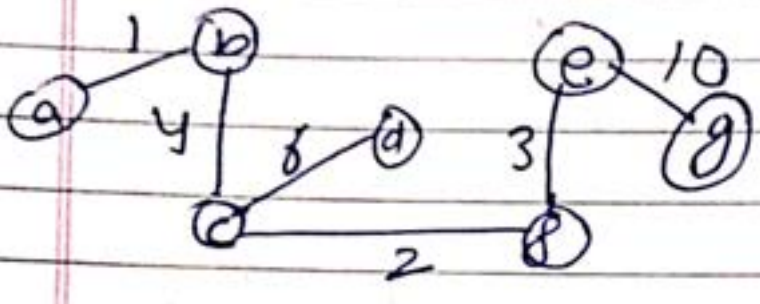




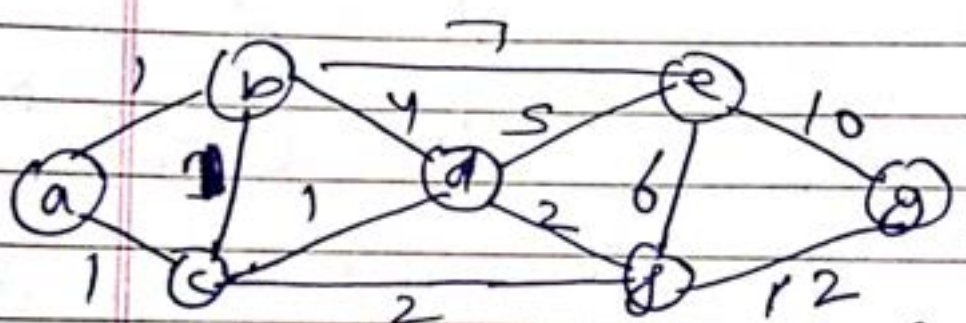
Apply Prim on



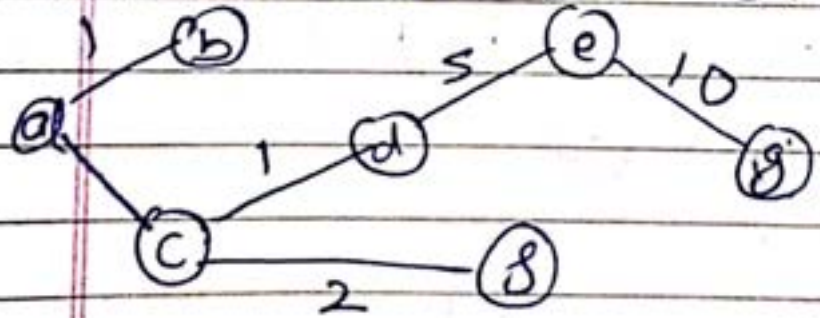
Ans



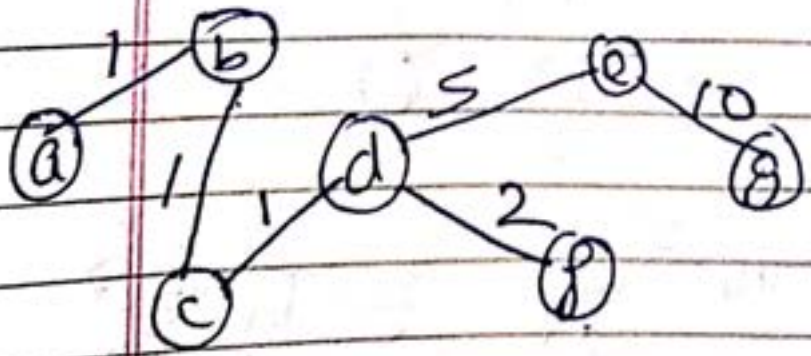
Q



Ans Many answers but cost (Sum) same



Am = 20

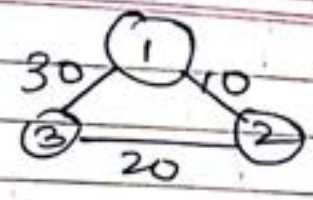


Scanned with CamScanner

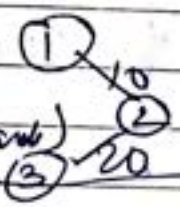
Apply Prim's

Date / Page 279

	1	2	3
1	0	10	30
2	10	0	20
3	30	20	∞

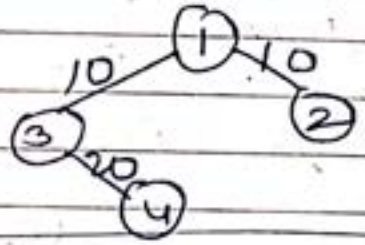


Ans



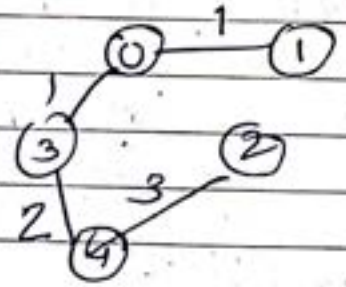
Let's see directly spanning tree

	1	2	3	4
1	0	10	10	50
2	10	0	40	30
3	10	40	0	20
4	50	30	20	0

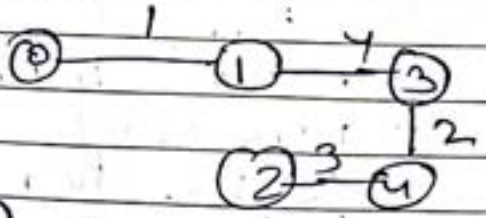


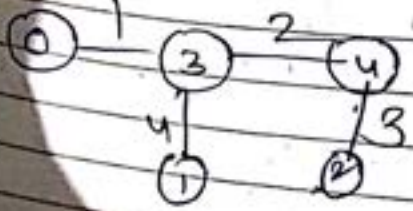
Date 2010

	0	1	2	3	4
0	0	1	8	1	4
1	1	0	12	4	9
2	8	12	0	7	3
3	1	4	7	0	2
4	4	9	3	2	0



0 as leaf



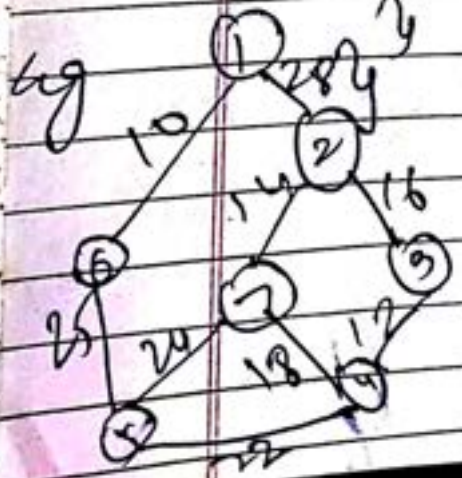


Scanned with CamScanner

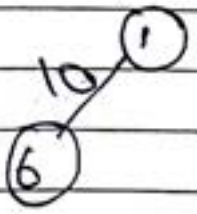
Not gate but for marks

Algorithm Prim (E, cost, n, t) {
// E is set of edges. cost is (n x n) adjacency matrix
// MST is computed & stored in
// array t[1:n-1, 1:2]
(n-1 edges)

1. Let (k,l) be an edge of min cost in E,
2) mincost = cost[k, l];
3) t[1,1] = k; t[1,2] = l;
4) for (i=1 to n)
5. if (cost[i,l] < cost[i,k]) then near[i]=l
else near[i]=k;
7) near[k] = near[l] = 0;
8) for (i=2 to n-1) {
9) let j be an index such that near[j] ≠ 0
10) cost[j, near[j]] is minimum,
11) t[i,1] = j
12) near[j] = 0;
13) for k=1 to n do
12) mincost = mincost + cost[j, near[j]];
13) near[j] = 0;
14) for k=1 to n do
15) if (near[k] ≠ 0 and (cost[k, near[k]] > cost[k, j]))
then near[k] = j;

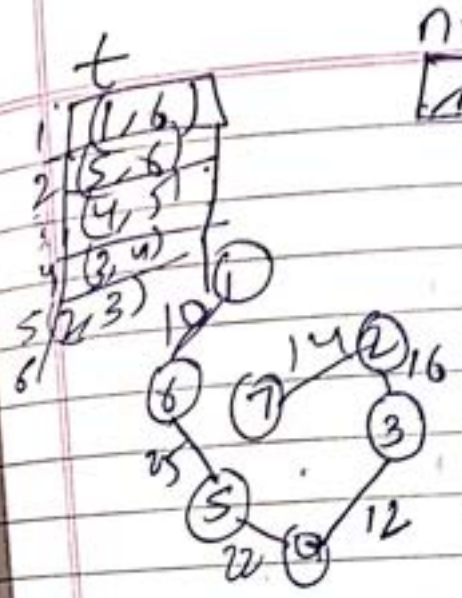


1)



Scanned with CamScanner

min cost = 10 + 25 + 22 + 12 + 16 = 85

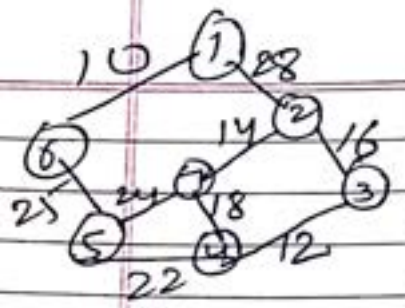


1) O(E)
4) O(n)
8) O(n),] O(n²)
9) O(n)
14) O(n)

```
MST_PRIM(G, wt, s)  // using min heap
1  for each vertex u ∈ G.vertices
       u.key = ∞
       u.Π = NIL
4  s.key = 0
5  Q = G.vertices
6  while (Q ≠ 0) {
       u = Extract_MIN(Q)
       for each vertex 'v' adjacent to 'u' {
           if v ∈ Q and cost(u,v) < v.key
               v.parent = u;
               v.key = w(u,v)
       }
```

Scanned with CamScanner

G → Graph
cost → matrix
r → root



	1	2	3	4	5	6	7
key	∞	∞	∞	∞	∞	∞	∞
π	N	N	N	N	N	N	N

Let's say $r = 0$, so $r.key = 0$

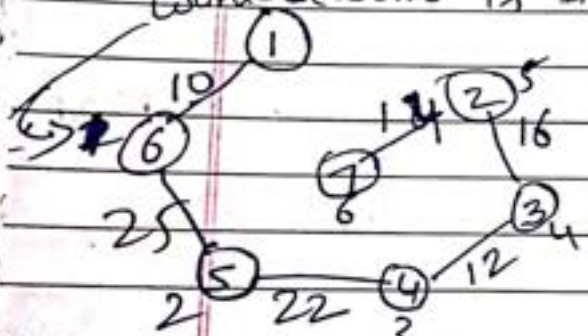
	1	2	3	4	5	6	7
key	0	28	12	22	25	10	24
π	N	5	4	3	2	1	5

Q = G.vertices, Q is a min-heap

u = 1 ∵ (key = 0 min)

count = 0 (count is # of times while executed)

(7 − 1 = 6)



$n = V$

1–3	$O(n)$
5	$O(n)$
6	$O(n)$
7	$O(\log n)$
11	$O(\log n)$
8	$O(n)$

$O(V^2 \log V)$

not right

if we do aggregate analysis $O(E)$

So Extract min $- O(V \log V)$

Decrease key $- O(E \log V)$

Build heap $- O(E)$

Total $= O(E \log V)$

Scanned with CamScanner

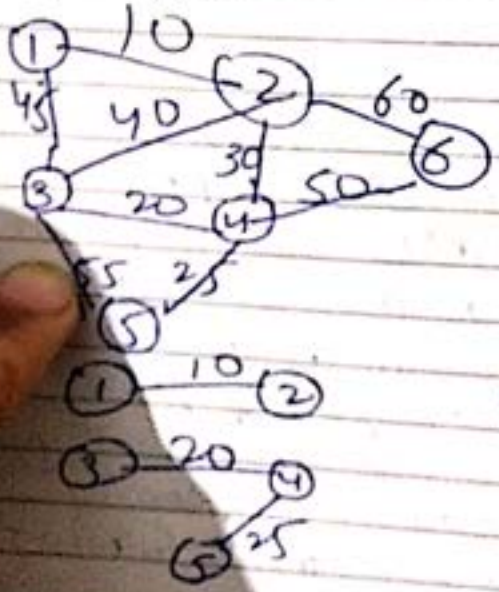
If we use fibonacci heap
$\log V \longrightarrow O(1)$
to decrease key

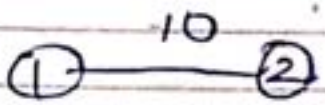
not for GATE

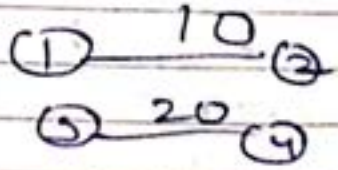
So heap Is $O(E \log V)$ better or $O(V^2)$

Dense	$E = O(V^2)$	$O(V^2 \log V)$,	$O(V^2)$ better
Sparse	$E = O(V)$	$O(V \log V)$ better	$O(V^2)$

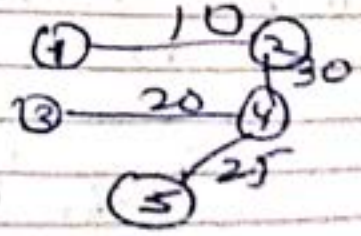
Kruskal's algo
(Graph can be disconnected in intermediate steps)

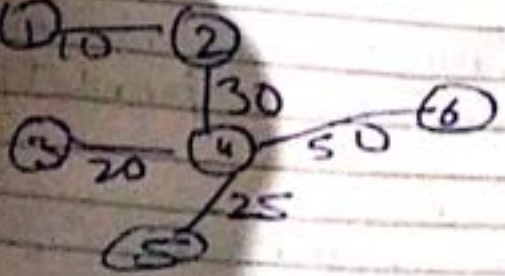


1)


2)


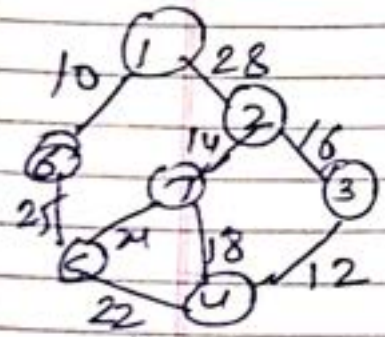
3)

4)


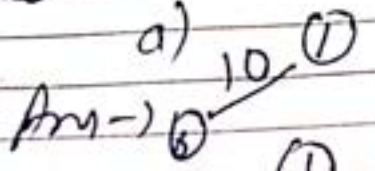
5)


Scanned with CamScanner

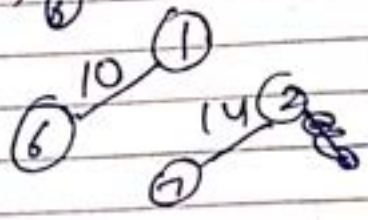
Kruskal's algo (Those min not added which will form cycle)



a) Ans-)

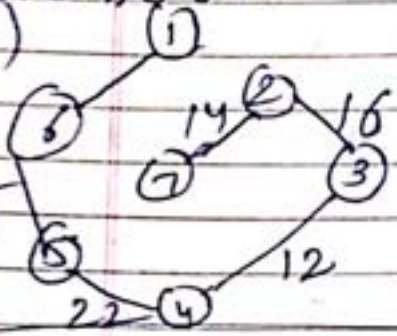


b)



and so on

c)



Gate 2000

Q) Let 'G' be an undirected connected graph with distinct edge weights. Let max_e be the edge with maximum weight and min_e be the edge with minimum weight. Which of the following is false?

a) Every min spanning tree of 'G' must contain min_e

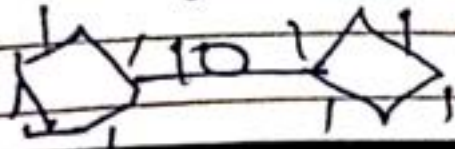
b) If max_e is in a minimum spanning tree, then its removal must disconnect 'G'

c) No minimum spanning tree contains max_e

d) 'G' has a unique min spanning tree.

Sol a, b, d are straightforward

Sol c is false, visualize b & c from

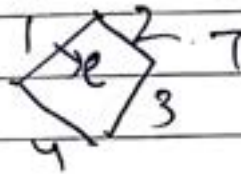


Gate 2007

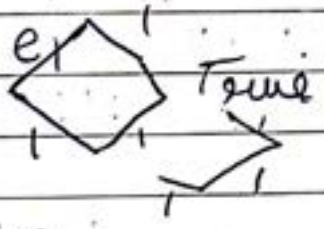
Q-1 Let 'w' be min weight among all weights in an undirected connected graph. Let 'e' be a specific edge of weight 'w'. Which of the following is FALSE

a) There is a min spanning tree containing 'e'
b) If 'e' is not in a min spanning tree 'T', then in the cycle formed by adding 'e' to 'T', all edges have the same weight
c) Every min spanning tree has an edge of weight 'w'
d) 'e' is present in every min spanning tree

Ans d) false

a) 

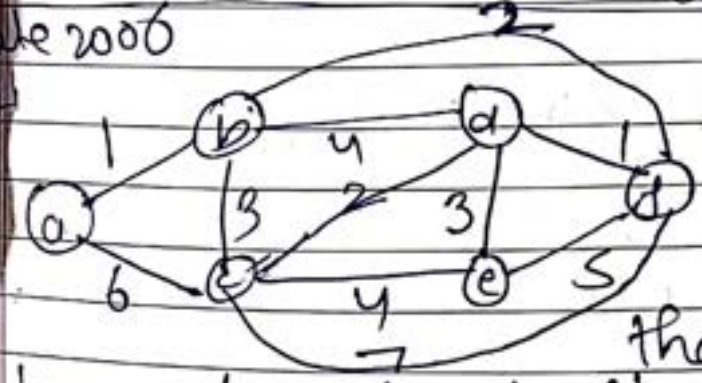
True

b) 

True

d) false.

Gate 2006

Q- 

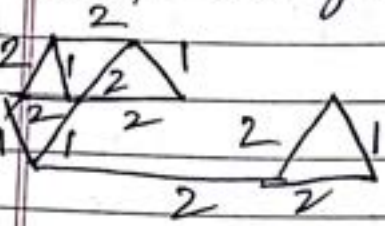
Which of the following can't be the sequence of edges added in that order to min spanning tree using kruskal's algorithm.

a) (a-b)(d-f)(b-f)(d-c)(d-e)
b) (a-b)(d-f)(d-c)(b-f)(d-e)
c) (d-f)(a-b)(d-c)(b-f)(d-e)
d) (d-f)(a-b)(b-f)(d-e)(d-c)

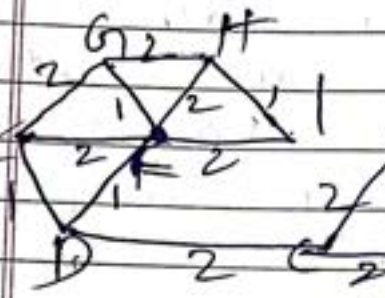
Scanned with CamScanner

2014

Q The # of min distinct spanning trees for the weighted graph below is —



Ans →

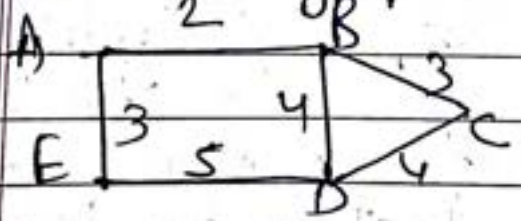


A → B mandatory
A → C & C → B ($^2C_1$)
C → D mandatory

F → G, D → E, D → F Mandatory
H → I mandatory
G H, HF, FI → $^3C_1$

So $3 \times 2 = 6$

Q Possible # of spanning trees for.



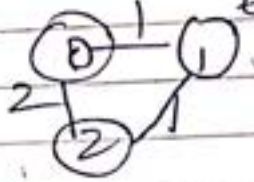
Ans

A ——— 2 ——— B Mandatory
B ——— 3 ——— C Mandatory
B → D C → D Choice $^2C_1$ Ans 2

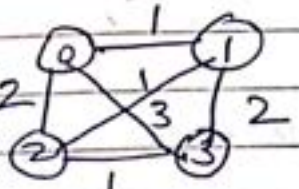
Scanned with CamScanner

Q1 A complete, undirected, weighted graph 'G' is given on the vertex of 0, 1, ..n-1 for any fixed 'n'. Draw the min spanning tree of G if
a) The weight of edge (U, V) is |U-V|
b) The weight of edge (U, V) is (U+V)

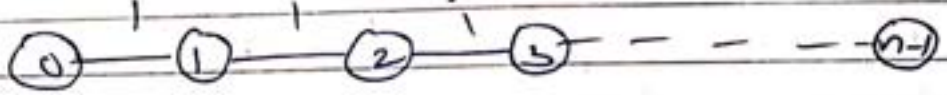
Ans let's say n=3



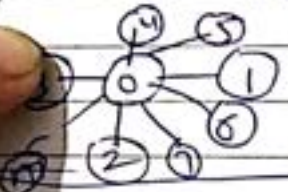
n=4



a)

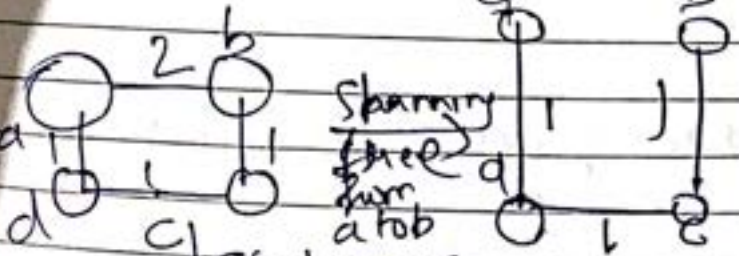


b) Pick all from 0 ; sum least



How shortest path & min spanning tree are different

Ans → Let's take an eg →



Shortest path



Scanned with CamScanner

relaxing an edge :



So here we relax $S \to u$ and $d(u) = 15$

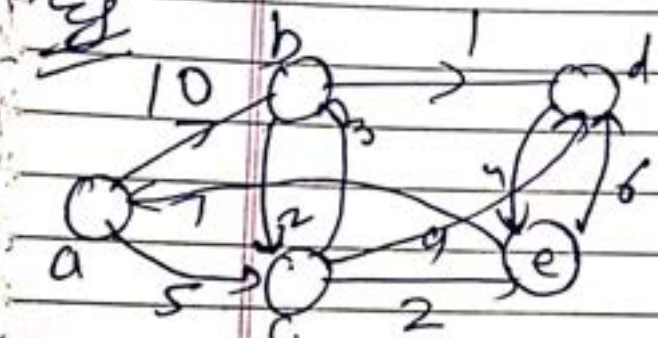
from S to V (shortest)

```
if ( d[u] > d(v) + c(v,u)        cost
     d[u] = d(v) + c(v,u)   → (i) to relax
```

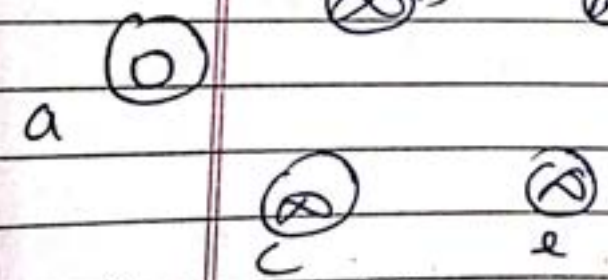
Also depends on DS

Dijkstra algorithm → not applied on -ve edge weights (works on some -ve & not on others)

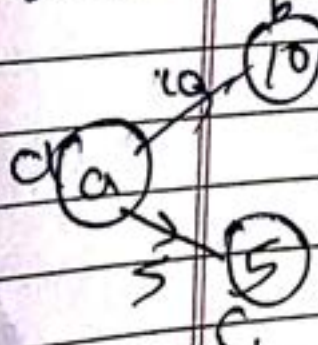
Eg



1) Pick up source & dist = 0 for source & ∞ for all other nodes



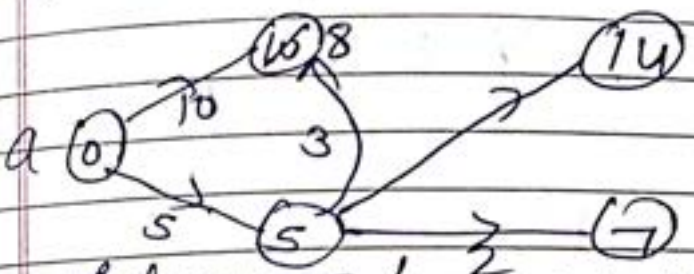
2) Relax all edges from a to other nodes



Scanned with CamScanner

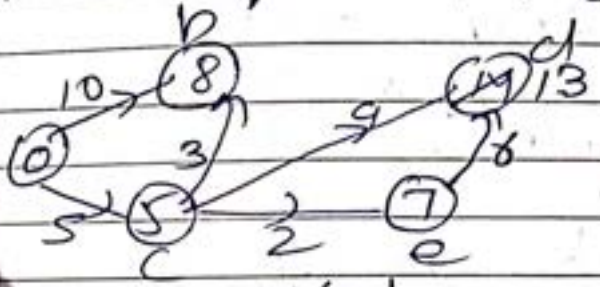
Believe a as dalbed

3) Now choose c (∵ min from b & c)
from c to others relax outgoing edges

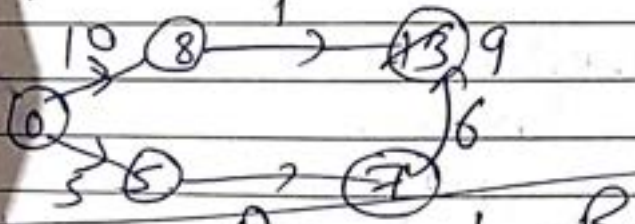


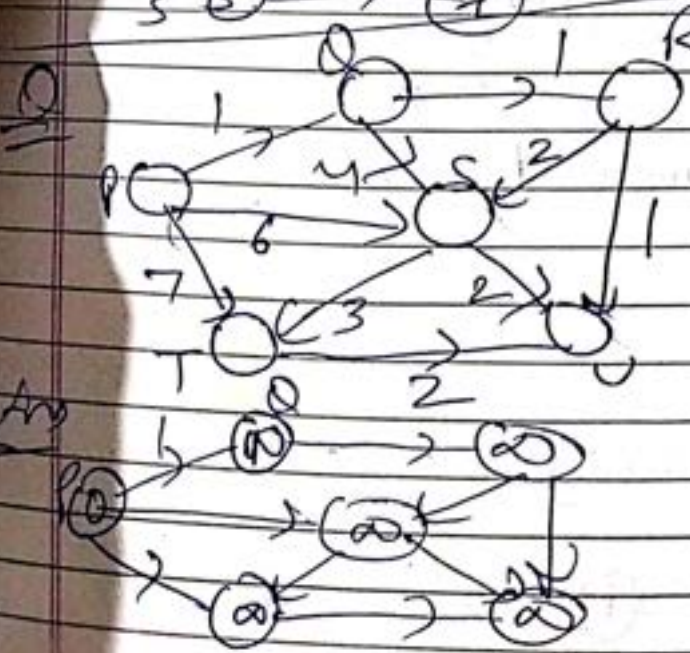
Believe a b c as dalbed

4) Pick up least i.e 7 (e)



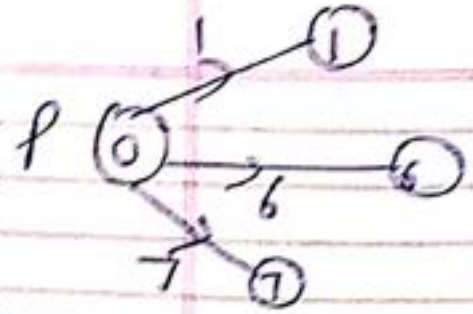
5) Pick b (8)





find order P is down

Scanned with CamScanner



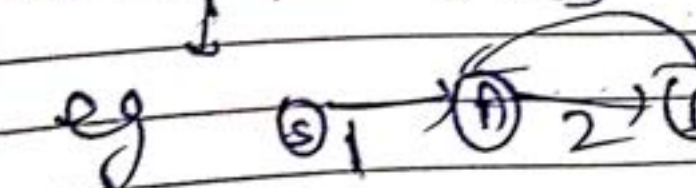
→ Do yourself
FORUST

Dijkstra (G, w, s)
1. Initialise-single-source (G, s) O(V)
2. S ← ∅
3. Q ← G.V (Build heap) ← O(V)
4. while Q ≠ ∅ O(V)
   u = Extract_min(Q) O(log V)
   S = S ∪ {u}
   for each vertex v ∈ G.Adj[u]
      Relax (u, v, w)

Decrease key − O(log V)
In worst case relax called everytime
So O(E) → O(E log V)
relax

Total O(V + V + V log V + E log V)
O(E log V)

When fail
1) disconnected
2) -ve weight cycles (−∞)

eg 

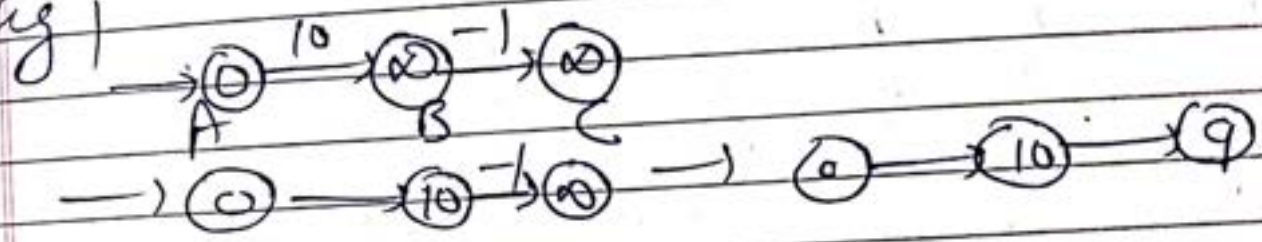
Date
Page 291

Algo

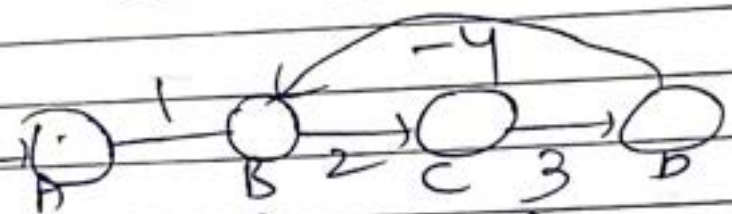
-ve cycle (X) | no-ve cycle (Dijkstra)

But Dijstra can't detect -ve cycles

Eg 1



2)



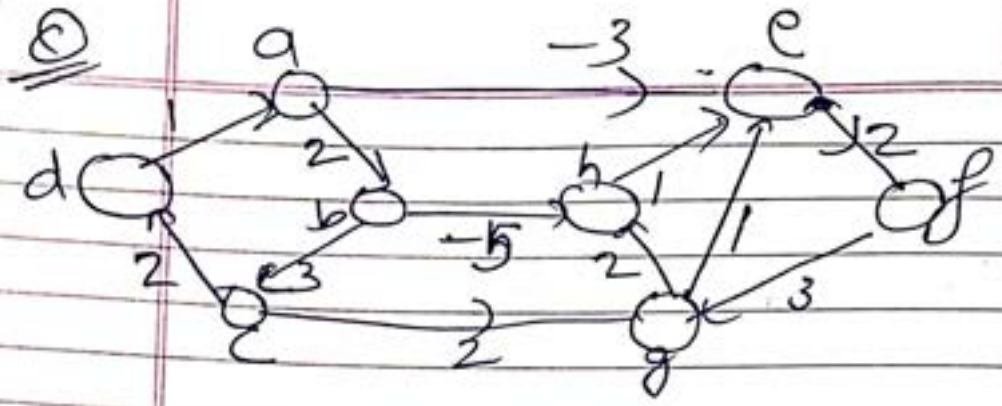
A	B	C	D
0	∞	∞	∞
	1	∞	∞
	1	3	∞
		3	∞
		3	6
		6	~~4~~

Ans is correct
from D → B
6 − 4 = 2
So wont happen
because B is at 1 but already

So in 2) Dijstra Succes
If −6 instead of −4 then fails

Scanned with CamScanner

Date
Page



	a	b	c	d	e	f	g	h
	0	∞	∞	∞	∞	∞	∞	∞
a		2	∞	∞	-3	∞	∞	∞
e		2	∞	∞		-1	∞	∞
f		2	∞	∞			2	∞
g		2	∞	∞				4
b			5	∞				-3
h			5	∞				
c				7				
d								

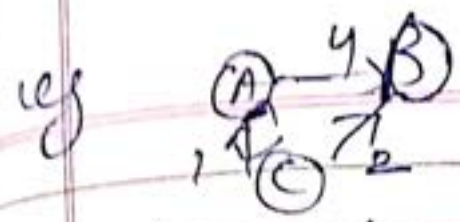
Sequence not unique here

Bellman-ford → No shortest path.
→ No (-∞ cycle)
→ Yes (SP)
↳ Shortest Path

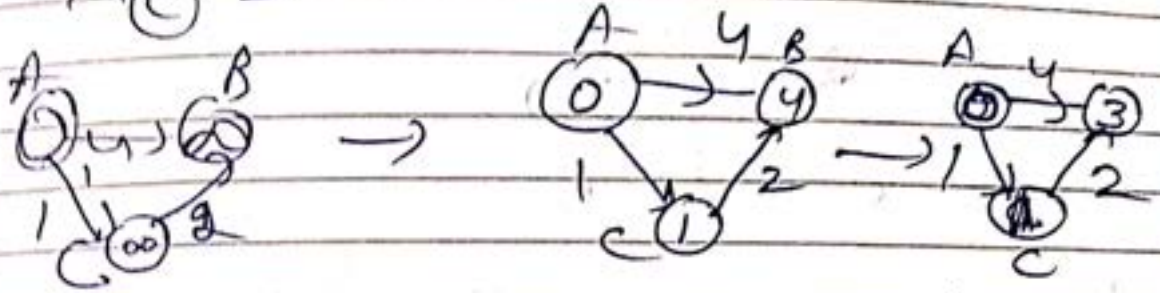
Time complexity better of Dijkstra
Real life → Dijkstra
BF → just a concept

BF → If graph has n nodes then SP is not more than n-1
ie ① → ② Not more than 2 edges

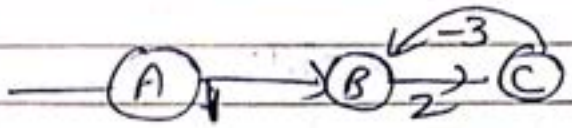
Scanned with CamScanner



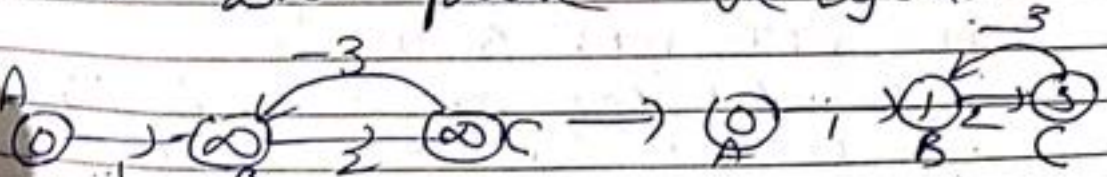
(Relax 2 times)



After including n-1 edges (relaxations) If you are still able to do a relaxation then (-ve cycle) (∵ +ve cycle cannot relax)



We will run for n-1 i.e 2 times and then we will relax for 1 more time and prove -ve cycle.



3rd relax



wrong (-ve cycle)

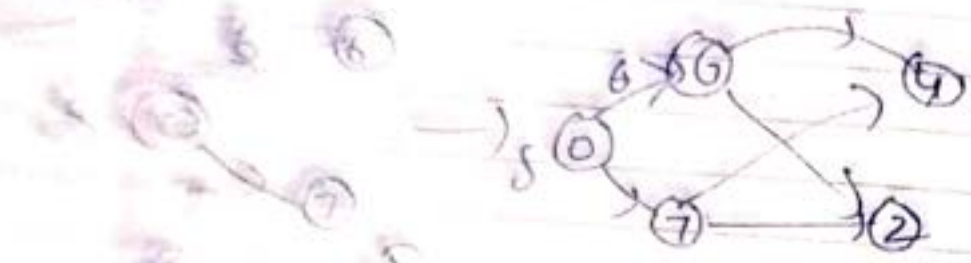
Running $(V-1)+1$ & E edges

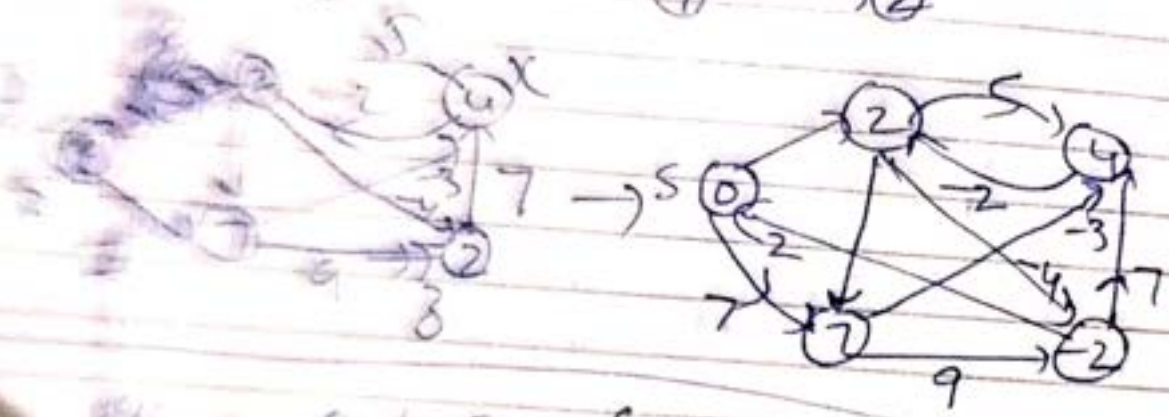
(VE) relax → we know last edge but have no need, we take away has also (1)

So $O(VE)$

Time complexity







BF MAIN_FORD(G, w, s){
Initialize single source (G, s) // O(V)
for i=1 to |G.V|-1 // (V-1) } O(V)
for each edge (u,v) ∈ G.E // (E) } O(E)
RELAX (u, v, w) // Array O(1) O(VE)
for each edge (u,v) ∈ G.E
if (v.d > u.d + w(u,v)) } O(E)
return FALSE
return TRUE
}
Time complexity = O(VE)

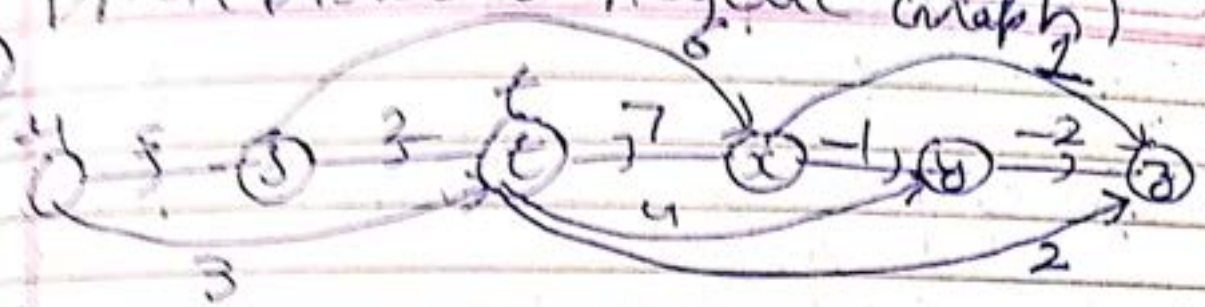
Scanned with CamScanner

↑ No cycle ! so we allow -ve wt edges

Date 1 / Page 295

DAG (Directed Acyclic Graph)

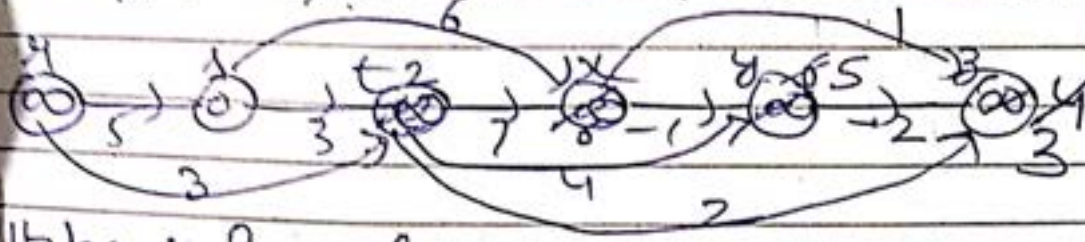
Q



Ans

1. Find Topological sorting
   ↳ Go from left to right in such a way that all directions are from L→R
   It takes O(V+E) to sort a DAG in topological order

2. S is start, so s = 0, all other ∞



(i) take u & relax
(ii) take s & relax
and so on

3. Time complexity:
   Topological − O(V+E)
   Every vertex, every adjacent edge (VE O(1) ✗ Take)
   ∴ every edge relaxed exactly once
   So O(E)
   Total O(V+E+E)
   = O(V+E)

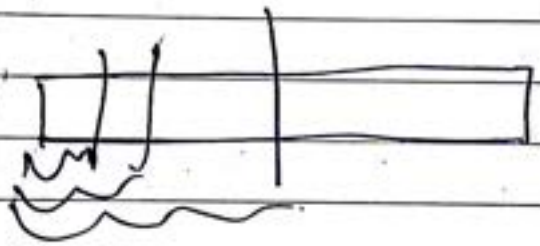
Scanned with CamScanner

DAG-Shortest-paths(G, w, s) {
1) Topologically sort the vertices of G - O(V+E)
2) Initialize-single-source (G, s) = O(V)
3) for each vertex u, taken in topologically sorted order
4) for each vertex v ∈ G.adj[u]
relax (u, v, w)
}
→ O(E)

Dynamic Programing (Bit similar to DAC but not greedy)

Programming mean - tables
Dynamic - We decide dynamically (Runtime)

DAC



DAC requires small subproblems (same) to solve them again & again but in DP we make a table & store the soln of those small problems so that we can fetch them again when required

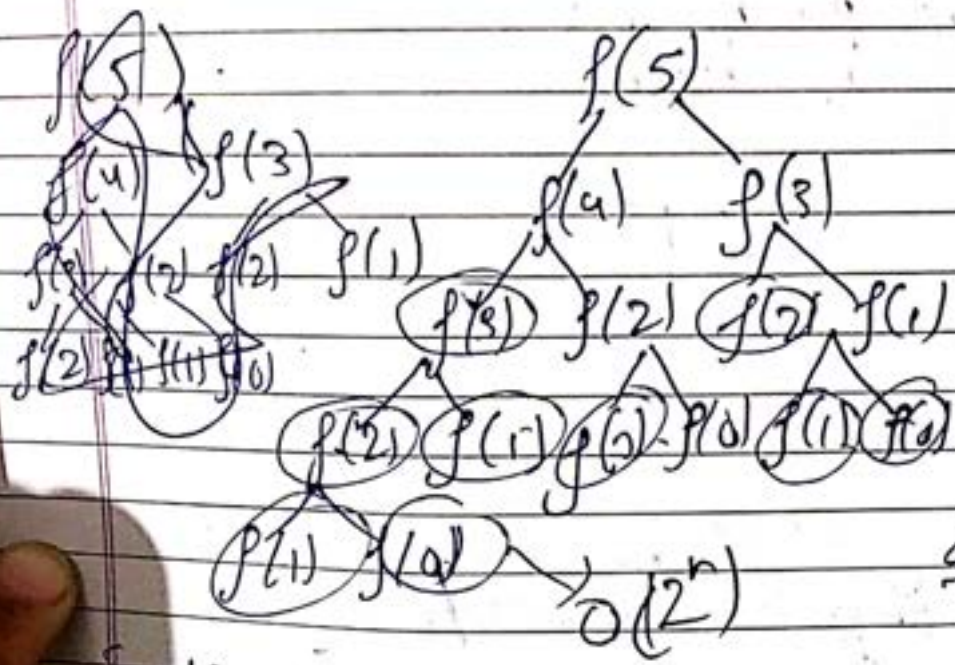
eg Fibonacci
$f(n) = f(n-1) + f(n-2)$; otherwise
$= 1$; $n = 1$
$= 0$; $n = 0$

n =	0	1	2	3	4	5	6
f(n) =	0	1	1	2	3	5	8

Scanned with CamScanner

```
f(n) {
  if (n==0)
    return 0;
  if (n == 1)
    return 1;
  return (f(n-1)+f(n-2));  — (1)
}
```

Let's see time taken



9 calls which are already called and we are wasting time

So if we maintain a table

2	3	4	5
f(2)	f(3)	f(4)	f(5)
2	3	5	8

T(n)

So instead of (1)

check the table T(n)

if T(n) is empty

return (f(n-1)+f(n-2));

Now O(n) - f(0, 1, 2, 3, 4, 5) all are called once

Scanned with CamScanner

Eg → we are lucky that in fibonacci there are overlapping problems, but in other like merge sort there are not, so we can't apply DP on such algos.

Matrix multiplication

A(3×2)

B(2×3)

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix}$$

$$C = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}_{3\times 3}$$

So order = 3 × 3 × 2 here

3 times (per row) × 3 times (per column) × 2 products individually

Generally = $\begin{bmatrix} & \end{bmatrix}_{P\times R} \begin{bmatrix} \\ \\ \end{bmatrix}_{R\times Q}$

$P \times Q \times R$

Scanned with CamScanner

Let's say we want to multiply
A, B, C

1 way $(A_{2\times1} \times B_{1\times2}) \; C_{2\times4} \rightarrow D_{2\times2} \times C_{2\times4}$ (4) $(2\times2\times4) = 16$

2 way $(A_{2\times1} \times (B_{1\times2} \times C_{2\times4})) \Rightarrow = 16$

$(A_{2\times1})$ ↳ 8 $(D_{1\times4})$ ↳ 8 , $8+8=16$

2nd way better

ABC . # of ways to parenthesize

(AB)C A(BC)

ABCD Assume $A_{1\times2}$ $B_{2\times1}$ $C_{1\times4}$ $D_{4\times1}$

A(BCD) (AB)(CD) (ABC)D

A(B(CD)) ((AB)C)D (A(BC))D

Let's see this cost

A(B(CD))

1×4×1 = 4 & resulting 1×1 = 4

B×(CD)$_{1\times1}$ = 1×2×1 = 2

lastly A(×) = 2

So 4+2+2 = 8

A((BC)D) ←

= 8+8+2 = 18

(AB)(CD)
2+4+1=7

(A(BC)D)
8+8+4=20

(A)B)()D
2+4+4=10

Best (AB)(CD) ← (7)

3 → 2 ways
4 → 5 ways
5 → 14 ways

ABCDE

A(BCDE) (AB)(CDE) (ABC)(DE) (ABCD)E
5 1×2 2×1 5

formula: $\frac{(2n)!}{(n+1)!n!}$ $n=3$ $= \frac{5\times 6}{6} = 5$ → for 4 matrix

$n=4$ → 14 → for 5 matrix
i.e put value of n for value of
$n=6$ → 42 → for 6 matrix

Interesting $\frac{(2n)!}{(n+1)!n!}$ is also formula to find out # of BST

2 things to note if we apply DP
1) optimal substructure & recursive
2) Overlapping problems required

1) Optimal substructure & recursive soln

Assume $A_1 A_2 \dots A_n$ are n matrices

$A_1 \to p_0 \times p_1$ $A_2 \to p_1 \times p_2$

$A_n \to p_{n-1} \times p_n$

$A_i \ (p_{i-1} \times p_i)$

let's assume parentheses

$A_i A_{i+1} \dots A_k)(A_{k+1} \dots A_j)$

$i \le k < j$

$(p_{i-1} \times p_k)$ $(p_k \times p_j)$

$A_{i \dots k}$ $A_{k+1 \dots j}$

So cost Associated $= p_{i-1} p_k p_j$ + cost of

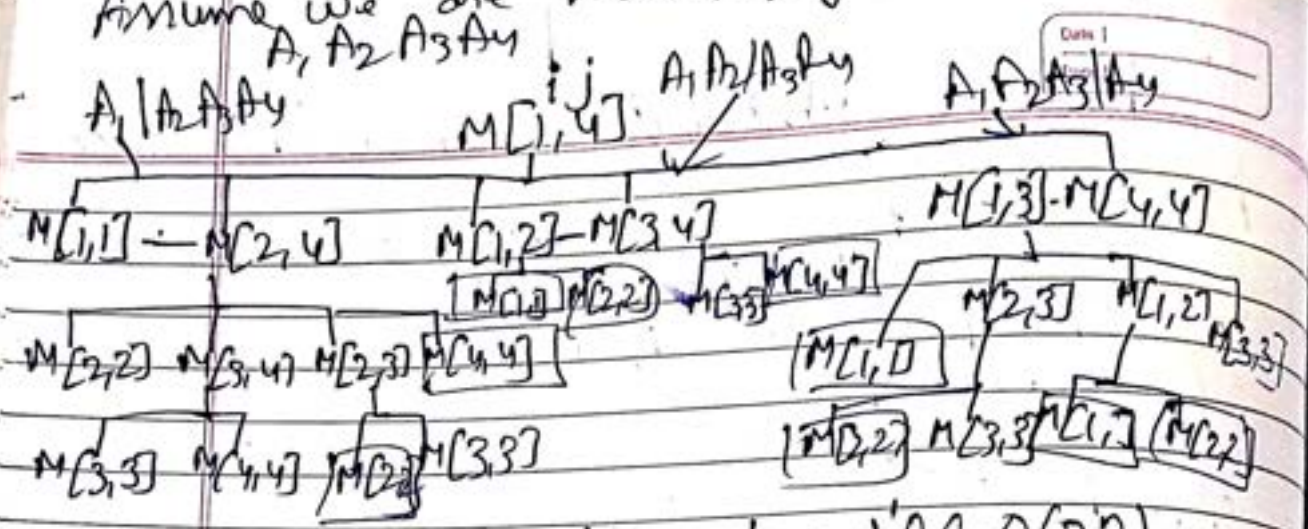
Now, let's see how to form recursive eqn

$m[i,j] \to$ min multiplications required if we parenthesize from i to j

$$m[i,j] = \begin{cases} 0 & i = j \\ \min_{i \le k < j} \{ m[i,k] + m[k+1,j] + p_{i-1} p_k p_j \} & \end{cases}$$

$m[i,k]$: $p_{i-1} \times p_k$; $m[k+1,j]$: $p_k \times p_j$

Assume we are multiplying 4 matrices
$A_1 A_2 A_3 A_4$



It is not binary tree but still $O(2^n)$ exponential
So we apply DP :

(2) Overlapping problems — Yes

M[1,1] — 4 times
M[2,2] — 5 times
and so on
So we are good to go for DP

So we get $O(n^3)$ — how let's see
2/9/20
Repeated calls n (11) (22) (33) (44) — 4 (Size 1)
n-1 (12) (23) (34) — 3 (Size 2)
n-2 (13) (14) — 2 (Size 3)
n-3 (14) — 1 (Size 4)

$n + (n-1) + (n-2) - - - + 1$
So $O(n^2)$ repeated problems
if $A_1 A_2 \sim A_n$

Scanned with CamScanner

To solve every problem O(n)
Total $O(n^3)$

Tree approach was top down

Another way (Bottom up approach)

A_1	A_2	A_3	A_4
1×2	2×1	1×4	4×1
p_0 p_1	p_1 p_2	p_2 p_3	p_3 p_4

(1,1) 0	(2,2) 0	(3,3) 0	(4,4) 0
(1,2) 2	(2,3) 7	(3,4) 9	
(1,3) 6	(2,4)		
(1,4) 7			

Now (1,3)

(1,1)

$(1,3) = \min\{ (1,1) + (2,3) + p_0p_1p_3 = 16,\ (1,2) + (3,3) + p_0p_2p_3 = 16 \}$

$(2,4) = \min\{ (2,2) + (3,4) + p_1p_2p_4 = 6,\ (2,3) + (4,4) + p_1p_3p_4 = 16 \}$

$(1,4) = \min\{ (1,1) + (2,4) + p_0p_1p_4 = 8,\ (1,2) + (3,4) + p_0p_2p_4 = 7,\ (1,3) + (4,4) + p_0p_3p_4 = 10 \}$

$O(n^2)$ entries
to compute 1,4 → 3 splits $O(n)$
Total = $O(n^3)$ by DP bottom up also
Space comp = $O(n^2)$

Scanned with CamScanner

Better: Understand by A_1 A_2 A_3 A_4
1×2 2×1 4×4 4×1
, 1 2 2 1 4 4 1

MATRIX-CHAIN(P) ← array p[1 2 1 4]

```
1   n = P.length - 1
2   let m[1..n, 1..n] and s[1..n-1, 2..n] be new tables
3   for i = 1 to n
4       m[i,i] = 0
5   for l = 2 to n // l is the chain length
6       for i = 1 to n-l+1
7           j = i+l-1
8           m[i,j] = ∞
9           for k = i to j-1
10              q = m[i,k] + m[k+1,j] + p_{i-1} p_k p_j
11              if q < m[i,j]
12                  m[i,j] = q
```

1. n = P.length - 1 ∴ P.length = 5 (for p_0, p_1, p_2, p_3, p_4) and matrices are 4
2. We don't need n×n matrix but still we have no other choice

(1,1)	(2,2)	(3,3)	(4,4)
(1,2)	(2,3)	(3,4)	
(1,3)	(2,4)		
(1,4)			

← actually half of n×n but no way to declare half so we declare full

5. m → used for split
5. In step 4. all are filled with 0 so we directly start from l = 2 (not 1)

Scanned with CamScanner

top-down
Memoisation :- Let's say we want (1,4)
now while traversing tree if
value is present not in table 305
then only we traverse the tree else not.
(Time complex (by Same)

A. Memoized_matrix_chain(p) {
1) n = p.length - 1
2) Let m[1..n, 1..n] be a new table.
3) for i = 1 to n
4) for j = 1 to n
5) m[i,j] = ∞
6) return LOOKUP_CHAIN(m, p, 1, n)
}

B. LOOKUP_CHAIN(m, p, i, j) {
1) if m[i,j] < ∞
2) return m[i,j]
3) if i == j
4) m[i,j] = 0
5) else for k = 1 to j - 1
6) q = LOOKUP_CHAIN(m, p, i, k)
+ LOOKUP_CHAIN(m, p, k+1, j)
+ p_{i-1} p_k p_j
7) if q < m[i,j]
8) m[i,j] = q
9) return m[i,j]

Time - O(n^3)
Space - O(n^2) for table + O(n) for stack

Q.

M_1 M_2 M_3 M_4
(10×100) (100×20) (20×5) (5×80)

Ans

11	22	33	44
0	0	0	0
12	23	34	
20000	10000	8000	
13	24		
15000	50000		
14			
19000			

(1,1)+(2,3) ← 3

(1,3) = min { (1,1)+(2,3)+5000 = 15000
(1,2)+(3,3)+1000 = 21000

(2,4) = min { (2,2)+(3,4)+16000 = 168000
(2,3)+(4,4)+40000 = 50000

(1,4) = min { (1,1)+(2,4)+80000 = 130000 (↓14)
(1,2)+(3,4)+16000 = 44000
(1,3)+(4,4)+4000 = 19000

(((11(23))4) ← Ans — 19000

Subsequence (Diff from substring)

RAVINDRA
1 2 3 4 5 6 7 8

Substrg. — RAVI VINI
(Contiguous)

{1,2,3,4} RAVI
{1,3,5,8} RUNA } Subsequence (as long as in order You can miss some characters)

Scanned with CamScanner

& 2 also subsequence

How many possible
→ Every character can 2 possibilities
0 & 1 (included or not included)

$$2 \times 2 \times 2 - - - n = 2^n$$

If 2 DNAs
$D_1 = AGCCTCAGT$
$D_2 = GCCT$
If $D_1 = D_2$ 100% similar
if $D_2 \subset D_1$ or vice versa very similar
→ We find largest common subsequence
to find similarity

eg $\begin{bmatrix} D_1 \\ D_2 \\ D_3 \end{bmatrix}$ (ACG)3 (ACGT)4
(GAAK) (5)
→ Very similar

Find largest common sum
→ RAVINDRA & AJAY → {A, A}

1 algo to find is RAVINDRA — m
and AJAY — n
and subsequences 2^n compare
each 2^m with n

So worst case O(n) Take 2 pointers
and increment only when
there is something common

Scanned with CamScanner

A = RAVINDRA
B = AJAY

1) find all subsequences of 'A'
2) find for each 'A' whether it is a subsequence of 'B'
3) find longest among common SS.

1) $O(2^m)$
2) $O(n 2^m)$
3) $O(2^m)$

We don't need to find subsequences of B ∵ they are already included in common part

But problem - exponential time complexity

Let's apply DP and check

1) Optimal substructure

x: x1 x2 -- xm
y: y1 y2 -- yn

Longest common subsequence
If xm, yn equal then xm-1, yn-1 and soon & find longest common SS

If xm, yn not equal remove xm & find LCSS of x1 -- xm y1 -- yn
or remove xn & find LCSS of x1 -- xm & y1 -- yn

2) Write a subsequence of [illegible]

$x_1\ x_2 \dots x_i$

$y_1\ y_2 \dots y_j$

Let's say $C[i,j]$ = length of LCS

$$C[i,j] = \begin{cases} 0, & \text{if } i=0 \text{ or } j=0 \\ 1 + C[i-1, j-1], & i,j>0 \text{ and } x_i = y_j \\ \max(C[i+1,j], C[i,j-1]), & i,j>0 \text{ and } x_i \neq y_j \end{cases}$$

Recursion tree & overlap subproblems

X = {A, B, A, B}

Y = {B, A, B, A} Best Case

C(4,4)

C(3,3)

C(2,2)

C(1,1)

C(0,0)

Let's see worst case

X = {A, A, A, A}

Y = {B, B, B, B} Y = anon

C(4,4)

C(4,3) C(3,4)

C(3,3) C(4,2) C(3,3) C(4,2)

P. T. O

C(4,4)

C(4,3) C(3,4)

C(3,3) C(4,2)

C(2,3) C(3,2)

C(1,3) C(2,2) C(1,2) C(2,1)

C(0,3) C(1,2) C(0,2) C(1,1) C(0,2) C(1,1) C(0,1) C(1,0)

C(0,2) C(1,1) C(0,1) C(1,0) C(0,1) C(1,0)

C(0,1) C(1,0)

$O(n+m)$ Space

$O(2^{n+m})$ time

~~$O(2^{m})$~~

For DP

1) Optimal Substructure
2) Overlapping problems

Let's see

C(3,3)

C(2,3) C(3,2)

C(1,3) C(2,2) C(2,2) C(3,1)

C(0,3) C(1,2) C(1,2) C(2,1) C(2,1) C(3,0)

C(0,2) C(1,1) C(0,2) C(1,1) C(2,0) C(1,1)

C(0,1) C(1,0) C(0,1) C(1,0) C(0,1) C(1,0)

~~Unique 9 #s (3,3) (3,2) (3,1) (3,0)~~

~~Overlapping~~

Unique (3,3) (3,2) (3,1)

(2,3) (2,2) (2,1)

(1,3) (1,2) (1,1)

(0,3) (0,2) (0,1)

(0,3) (0,2) (0,1)

Scanned with CamScanner

	0	1	2	3
0	00	01	02	03
1	10	11	12	13
2	20	21	22	23
3	30	31	32	33

○ → 0 diag
Now to compute 11 00, 01, 10 must be computed

and 11 for every other element it should hold true.

Here we can only compute 11 after all 0's if you can see & after that only 12 13 21 22 and 10 etc 12 21 31 and soon
Row wise
column wise

Time complexity → O(mn) = space comp

Now above was bottom up
So for Top down we apply recursive fn and compute 33 first & so on. (known as top down) Time comp & O(mn) = space comp

But for
X = {A, A, A, A}
Y = {A, A, A, A}

Time = O(n)
Space = O(n)

C(4,4)
↓
1 + C(3,3)
↓
1 + C(2,2)
↓
1 + C(1,1)
↓
C(0,0)

Scanned with CamScanner

Wont Corp

X = [A, A, A, A]

Y = [B, B, B, B]

((4,4)

((4,3) ((3,4)

((3,2) ((4,2)

((2,3) ((3,2)

((4,3) ((2,2) ((3,1) ((2,2)

((0,3) ((1,2)

((0,2) ((1,1)

((0,1) ((1,0)

Ignore above repeated

```
    0    1    2    3
0 | 00 | 01 | 02 | 03 |
1 | 10 | 11 | 12 | 13 |
2 | 20 | 21 | 22 | 23 |
3 | 30 | 31 | 32 | 33 |
```

([n,m]

([n-1,m]

([n-1,m-1]

([n-2,m-1]

([0,1]

([1,0]

O(n+m) space

$O(2^{n+m})$ Time

X(AABB)

Y(ACA)

Let's solve

		A	C	A
	0	1	2	3
A 0	0	0	0	0
A 1	0	1	1	1
B 2	0	1	1	2
B 3	0	1	1	2

Ans

due to not equal

if match increment by 1 (if row major)

if no match check A[i-1,j] A[i,j-1]

And take higher

Scanned with CamScanner

Date: Page: 313

X = {A, B, C, B, D, A, B} (1 2 3 4 5 6 7)

Y = {B, D, C, A, B, A} (1 2 3 4 5 6)

		B (1)	D (2)	C (3)	A (4)	B (5)	A (6)
	0	0	0	0	0	0	0
D (1)	0	0	0	0	1	1	1
B (2)	0	1	1	1	1	2	2
C (3)	0	1	1	2	2	2	2
B (4)	0	1	1	2	2	3	3
D (5)	0	1	2	2	2	3	3
A (6)	0	1	2	2	3	3	4
B (7)	0	1	2	2	3	4	4

BCBA

Now, how to get the matching SS
2 paths from corner ↑ ← so just traverse & write all with match & use 1 once only 2 are only 3 once only ↖ 10 or.

```
LCS (X, Y) {                    (Bottom up)
m = X.length
n = Y.length
Let C[0...m, 0...n] be a new table
for i = 1 to m
    C[i, 0] = 0
for j = 0 to n
    C[0, j] = 0
for i = 1 to m
    for j = 1 to n
        if xi == yj
            C[i, j] = C[i-1, j-1] + 1
        else
            C[i, j] = max(C[i-1, j], C[i, j-1])
return C[i, j]
```

Time → O(mn)

Space → O(mn)

If Top down memoization used.
→ O(mn) + stack O(m+n)

Gate 2014.

A = pqpqrr

B = pqrqrp find x+10y

x → length of LCS

y → # of LCS possible

Ans

A = 1 2 3 4 5
    p q p q r

B = 1 2 3 4 5 6
    p q r q r p

B \ A	0	p	q	p	q	r	r
0	0	0	0	0	0	0	0
p	0	1	1	1	1	1	1
q	0	1	2	2	2	2	2
r	0	1	2	2	2	3	3
q	0	1	2	3	3	3	3
r	0	1	2	3	4	4	4
p	0	1	2	3	4	4	4

x = 4

1) pqrr
2) qpqr
3) qpqr

If x+10y = 34

Scanned with CamScanner

Gate 2009 315

Q We are given 2 sequences $x[m]$ & $Y[n]$ of length m & n respectively with indices of x & y starting from 0. $l(i,j)$ is LCS of $x[m]$ and $Y[n]$ is given below

$l(i,j) = 0$, if either $i=0$ & $j=0$
$= expr1$, if $i,j > 0$ and $x[i-1] = Y[j-1]$
$= expr2$, if $x[i-1] \neq Y[j-1]$, $i,j > 0$

Ans Now 0 index, not 1 index.

So $expr1 = 1 + l[i-1][j-1]$

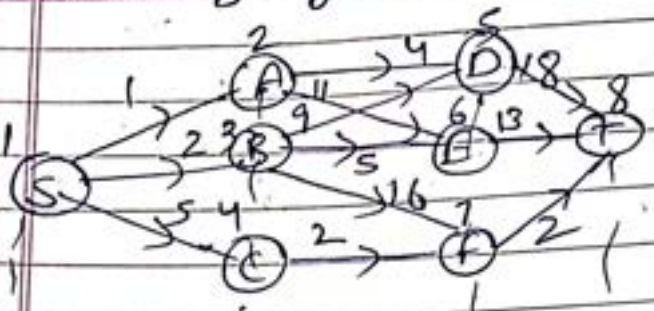
$expr2 = \max(l[i-1,j], l[i,j-1])$

Q $l[i,j]$ is computed by DP using an array $L[M,N]$ where $m = m+1$ and $n = n+1$, if $L[i,j] = l(i,j)$

1) All elements of L should be initialized to 0.
2) The values of $l(i,j)$ may be computed in row major order.
3) The value of $L(i,j)$ can't be computed in either row major order or column major order
$L[p,q]$ needs to be computed before $L[r,s]$ if either $p < r$ or $q < s$

∴ both $p < r$ or $q < s$
Not one of them

Multi-stage graph

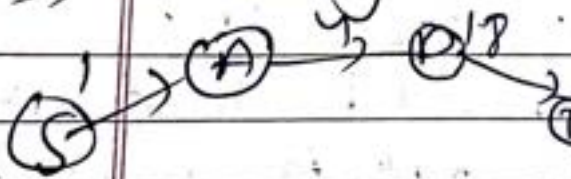
eg 

4 stages

Property – 1) B/w 1 stage no edges b/w Vertices eg A → B → B ↛ A etc.

2) At stage 1 only one node S (Start)
3) At last stage only one node T (Target)
4) We have to find Shortest path b/w S & T

Not Preferred Ways
NP 1) Traverse all paths & find least
NP 2) Dijkstra (O(E log V))
3) Greedy



= 23

but best ans = 9 S → C → F → T

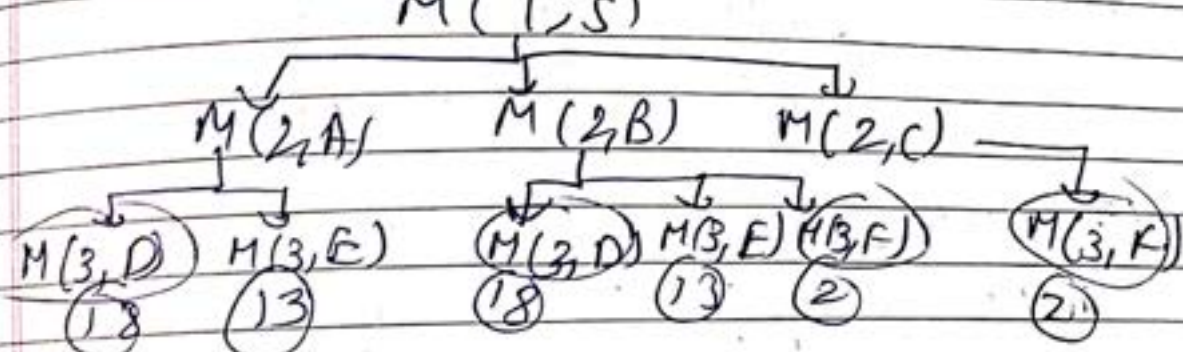
Fails

4) Dynamic Programming
→ Substructure & recursive eqn
→ overlapping subproblem

Scanned with CamScanner

Stage

$M(1,S) = \min \{ S \to A + M(2,A),\ S \to B + M(2,B),\ S \to C + M(3,C) \}$



we stop at $M(N-1, i) = i \to T$

Time — exponential (2^k) k = height

But there are overlapping subproblems

So we use DP

Bottom up DP approach

Label all nodes in increasing order & higher layer get higher number.

8 nodes, each 7 nodes

1	2	3	4	5	6	7	8
9	22	18	4	18	13	2	0
(7)	(6)	(5)	(4)	(3)	(2)	(1)	(1)

$T[i] = \min_{j=(i+1) \text{ to } n} \{ cost(i,j) + T[j] \}$

$T[5] = \min \{ cost(5,6) + T[6],\ cost(5,7) + T[7],\ cost(5,8) + T[8] \}$

$T[4] = c(4,7) + T[7] = 4$

$T[3] = \min \begin{cases} c(3,5) + T[5] = 22 \\ c(3,6) + T[6] = 18 \\ c(3,7) + T[7] = 12 \end{cases}$

$T[2] = \min \begin{cases} c(2,5) + T[5] = 22 \\ c(2,6) + T[6] = 28 \end{cases}$

$T[1] = \min \begin{cases} c(1,2) + T[2] = 23 \\ c(1,3) + T[3] = 20 \\ c(1,4) + T[4] = 9 \end{cases}$

Time complexity = $O(e)\frac{(n-1)(n)}{2} = O(V^2)$

$= O(E)$

Difference b/w fractional & 0-1 knapsack

Knapsack

Fractional (We can get fraction)

0/1 (We can't take the item by weight we have to take whole item as it is we don't we, partial or fractional item not allowed)

Let's solve below given by 0/1 knapsack (Do it on the same way)

0-Capacity = 6

objects	1	2	3
weight	1	2	4
profit	10	12	28

Now if you consider you can't use object 2 for more

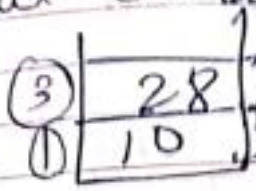
So ans is

$\frac{1}{3} \times 12$

$= 4$

$10 + 6 + 4$

but what greedy will do is



Profit = 38 not max

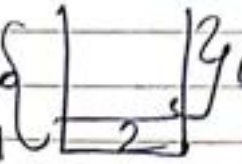
So greedy fails in this scenario.
So we go for DP.

But let's see by 1 more example why greedy fails.

→ Assume Capacity W.

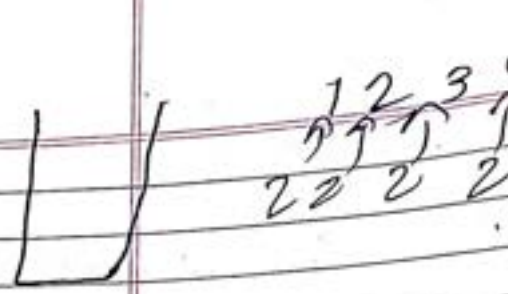
	1	2
Profit	2	W
Weight	1	W
P/W	2	1

So greedy choose obj 1 and knapsack is



So greedy fails, eg W=10 so max profit = 10 but it gives only 2 and also 8 units of knapsack are wasted. So we need DP

Scanned with CamScanner



2 choices either included or not

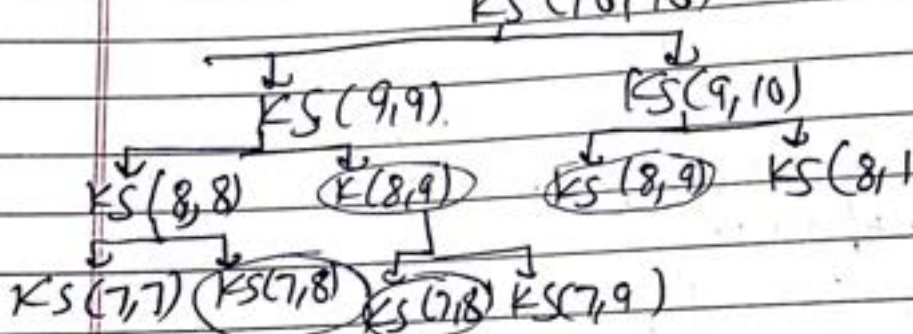
So $O(2^n)$

let's try to write recursive eq^n

$$KS(i,w) = \begin{cases} \max(P_i + KS(i-1, w-w_i),\ KS(i-1,w)) & ; i=0 \text{ or } w=0 \\ KS(i-1,w) & ; w_i > w \end{cases}$$

(# elements, total capacity)

eg KS (10,10)



Depth: O(n) So $O(2^n)$

No of unique subproblems = 10 x 10

= n x w

So create a table of size n x w & keep on filling entries

also we will make table size of n+1 x w+1 to give space for index 0 also

So O(nw)

Scanned with CamScanner

C = 6

	1	2	3
w	1	2	4
p	10	12	22

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	12	22	22	22	22
3	0	10	12	22	28	32	34

$KS(1,1) = \max\{10 + KS(0,0) = 10,\ KS(0,1) = 0\}$

$KS(1,2) = \max\{10 + KS(0,1) = 10,\ KS(0,2)\}$

$K(2,1) = K(1,1)$ (∵ cost + pw wt = 2 > cap = 1)

$K(2,2) = \begin{cases} p_2 + K(1,0) = 12 \\ K(1,2) = 10 \end{cases}$

$K(2,3) = \begin{cases} p_2 + K(1,1) = 22 \\ K(1,3) = 10 \end{cases}$

$K(2,4) = K(2,5) = K(2,6) = 22$

$K(3,1) = K(2,1)$

$K(3,2) = K(2,2)$

$K(3,4) = \begin{cases} p_3 + K(2,0) = 28 \\ K(2,4) = 22 \end{cases}$

$K(3,5) = \begin{cases} p_3 + K(2,1) = 32 \\ K(2,5) = 22 \end{cases}$

$K(3,6) = \begin{cases} p_3 + K(2,2) = 40 \\ K(2,6) = 22 \end{cases}$

Scanned with CamScanner

Time Complexity = $O(nw)$

space = $O(nw)$

If $w = 2^n$ Then $O(n2^n)$

So then use Brute force only not DP

$O(2^n)$

So $T(n) = \min\{O(2^n), O(nw)\}$

Depends on w

Algorithm

I/P : $\{w_1, w_2, w_3 \ldots w_n\}$, C, $\{P_1, P_2 \ldots P_n\}$ (capacity(or)W)

O/P : $T[n, C]$

for $i = 0$ to C do

$T[0, i] = 0$

for $i = 1$ to n do

$T[i, 0] = 0$

for $j = 1$ to C do

if $(w_i \leq j)$ and $T[i-1, j-w_i] + P_i > T[i, j]$

then $T[i, j] = T[i-1, j-w_i] + P_i$

else $T[i, j] = T[i-1, j]$

}

Scanned with CamScanner

Subset Sum.

Given $\{a_1, a_2 \dots a_n\}$ is there any subset whose sum can be "w"

eg: $\{6, 2, 3, 1\}$ $w = 5$

Yes $2+3=5$

Greedy fails because we don't know we need max. benefit (sum) or min. contribution etc.

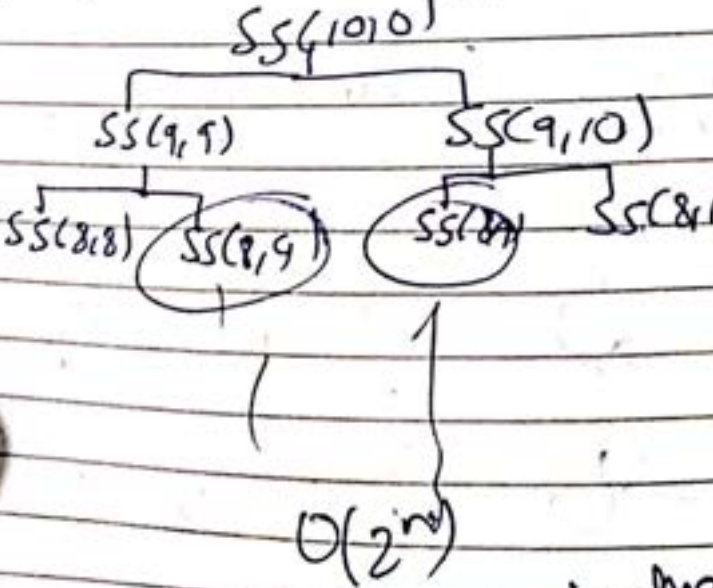
Brute force $= 2^n$ (2 choices of every elem)

Let's use DP

1) Substructure

$$SS(i,s) = \begin{cases} \text{False}; & i=0, s \neq 0 \\ \text{True}; & s=0 \\ SS(i-1, s-a_i) \text{ or } SS(i-1, s) & j > a_i \\ SS(i-1, s); & j < a_i \end{cases}$$

eg: $SS(10, 10)$



$O(2^n)$

Overlapping problems

$SS(n, w)$

$O(n, w)$

exactly like knapsack

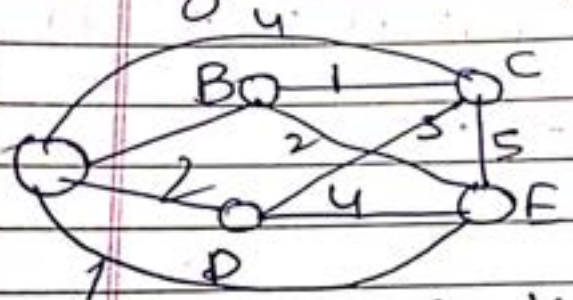
eg {6, 3, 2, 5} W = 5
add 1 extra row | extra col

	0	1	2	3	4	5
0	T	F	F	F	F	F
1	T	F	F	F	F	F
2	T	F	F	T	F	F
3	T	F	T	T	F	T
4	T	T	T	T	F	T

SS(1,1) = S(0,1) and so on Elem wt > Sum
SS(2,1) → false

Time Complexity = O(nW)

Travelling Saleman Problem



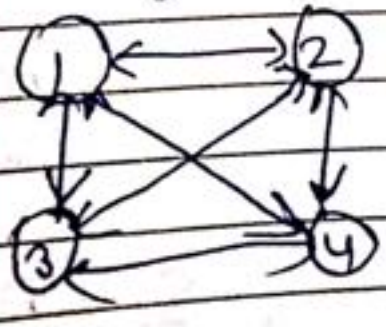
Traveller is at A
He has to travel to different cities and return to A and in min cost

A B C D E A
A C B D E A

⇒ O(n!)
P-S O(n!) >> O(2^n)

Let's see if DP helps

eg



	1	2	3	4
1	0	1	2	3
2	1	0	4	2
3	1	2	0	5
4	3	4	1	0

Scanned with CamScanner

start from 1 and end at 1

visit 2,3,4 on way

Formula

$$T(1,\{2,3,4\}) = \min \begin{cases} (1,2) + T(2,\{3,4\}) \\ 2(1,3) + T(3,\{2,4\}) \\ 3(1,4) + T(4,\{2,3\}) \end{cases}$$

go back to 1 not 2 not 3

$= 3+5 = 8$

$= 9$

$= 7$

$$T(2,\{3,4\}) = \min \begin{cases} (2,3) + T(3,\{4\}) = 2 \\ (3,4) + T(4,\{3\}) = 4 \end{cases}$$

$$T(3,\{2,4\}) = \min \begin{cases} (3,2) + T(2,\{4\}) = 5 \\ (3,4) + T(4,\{2\}) = 5 \end{cases}$$

$$T(4,\{2,3\}) = \min \begin{cases} (4,2) + T(2,\{3\}) = 5 \\ (4,3) + T(3,\{2\}) = \end{cases}$$

$T(3,\{4\}) = (3,4) + T(4,\phi) = 8$

$T(4,\{3\}) = (4,3) + T(3,\phi) = 2$

$T(2,\{4\}) = (2,4) + T(4,\phi) = 5$

$T(4,\{2\}) = (4,2) + T(2,\phi) = 5$

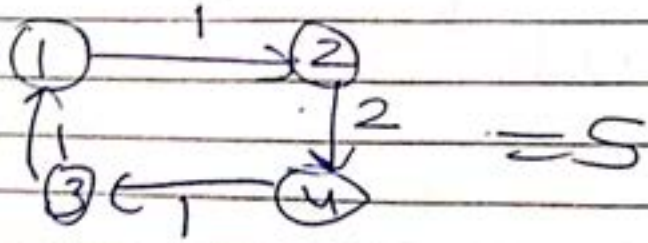
$T(2,\{3\}) = (2,3) + T(3,\phi) = 5$

$T(3,\{2\}) = (3,2) + T(2,\phi) = 3$

$T(2,\phi) = (2,1) = 1$

$T(3,\phi) = (3,1) = 1$

$T(4,\phi) = (4,1) = 3$



$= 5$

Scanned with CamScanner

Starting → Set of vertices (total between 0 and 3)

$$T(i,S) = \min_{j \in S} \{ (i,j) + T(j, S-\{j\}) \}; \quad S \neq \phi$$

$$(i,1); \quad S = \phi$$

$T(1, \{2,3,4\})$

$T(2,\{3,4\})$ $T(3,\{2,4\})$ $T(4,\{2,3\})$

$T(3,\{4\})$ $T(4,\{3\})$ $T(2,\{4\})$ $T(4,\{2\})$ $T(2,\{3\})$ $T(3,\{2\})$

$T(4,\phi)$ $T(3,\phi)$ $T(4,\phi)$ $T(4,\phi)$ $T(3,\phi)$ $T(2,\phi)$

$T(i,S) \rightarrow$ (n-2) at first level, (n-3) 2nd level

n-1 value at every level

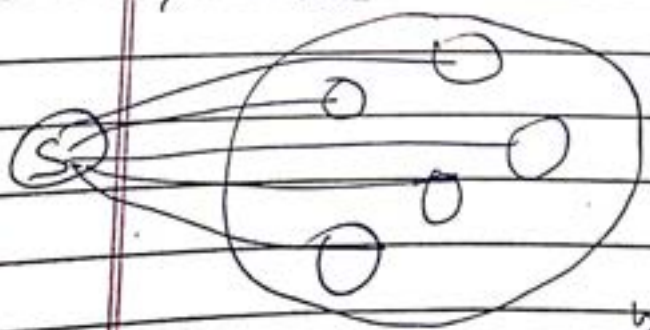
Total unique subproblems = $(n-1)2^{n-2} \rightarrow O(n^2 2^n)$

~~$i \cdot (n-1) \times (n) = S$~~

~~$= 3 \times 4$~~

i.e

Till now we have seen we have source node S and



we find shortest path from S to all other node

But, what if we want to find shortest path from every node to every other node.

OP fails totally fails

Date
Page 327

For one node Sorely Dijkstra $= O(E \log V)$
for all nodes Dijkstra $= O(VE \log V)$
$= O(V^3 \log V)$

But if -ve wt, then we go for
Bellman-ford algo
for sorely $O(VE)$
all nodes $O(V^2E) = O(V^2V^2) = O(V^4)$

P.S Dijkstra & Bellman ford are greedy algos, let's try DP now

Substructure
$V = \{1, 2, \ldots n\}$
$i, j \in V$
d_{ij}^{k} i.e $\{1, 2, \ldots k\}$ $= \ldots$ "p"

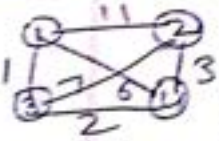
p — not using k $\left\{ d_{ij}^{k-1} \right.$
$\left. d_{ij}^{k-1} + d_{jk}^{k-1} \right\}$ $k \neq 0$
k used in path

$p_1 - p_2$

w_{ij} $k = 0$ (remaining vertex)

Scanned with CamScanner

4)



D^0 = (1st node already present) + 1 edge

	1	2	3	4
1	0	11	1	6
2	11	0	7	3
3	1	7	0	2
4	6	3	2	0

D^0 = 1st row + 1st column

$(2,3) = 7$

$2 \to 1 + 1 \to 3$

$11 + 1 = 12$ ✗

$(2,1) = 3$ other ✗

$(3,2) = 7$ $3 \to 1 + 1 \to 2$

$1 + 11 \to 12$

P.S → Can't use $3 \to 4 + 4 \to 2$ we can only use previous values available in table, that's what D^0 means.

Ind → D^1 Means only containing vertex 1 etc.

Now, let's solve D^2 → Including 1 & 2

D^2	1	2	3	4
1	0	11	1	6
2	11	0	7	3
3	1	7	0	2
4	6	3	2	0

D^3	1	2	3	4
1	0	6	1	3
2	8	0	7	3
3	1	7	0	2
4	3	3	2	0

$(1,2) = (11, 1 \to 3 + 3 \to 2$ from $D^2)$

$(1,4) = (6, 1 \to 3 + 3 \to 4$

$6, 1 \to 3$ ✓

$(2,3) \to (3, 2 \to 3 + 3 \to 4)$

$(1,2) = (8, 1 \to 3 \to 4 \to 2$

$(2,3) = (7, 2 \to 1 \to 4 \to 3$

$3 \to 2$ ✓

D^4	1	2	3	4
1	0	6	1	3
2	6	0	5	3
3	1	5	0	2
4	3	3	2	0

Ans

Scanned with CamScanner

Time & Space Comp.

Date: / Page: 329

Subproblems $\Omega(n^2) = O(n^3)$
↳ number of matrices of $O(n^2)$

Space complexity → At simultaneous situation we don't need > $2n^2$ so $O(n^2)$

Algo

```
FLOYD_WARSHALL(W)
1) n = W.rows
2) D^0 = W
3) for k = 1 to n
4)     let D^(k) = d(ij^k) be a new n×n matrix
5)     for i = 1 to n
6)         for j = 1 to n
7)             d_ij^(k) = min(d_ij^(k-1), d_ik^(k-1) + d_kj^(k-1))
8) return D^(n)
```

NP-completeness

Problems

Decidable problems (algo exists)

Intractable
1) If ∃ at least one polynomial bound algorithm $O(n^k)$

Intractable
2) If a problem is not tractable it is intractable
$\therefore O(n^{\log n}), O(c^n)$

Undecidable (no algo) like halting problem of TM
Travelling salesman

Scanned with CamScanner

P → Problems.



→ undecidable
→ decidable
→ polynomial
→ Non-P

Empty & we don't know in future that they can be optimized.

Eg: Real life

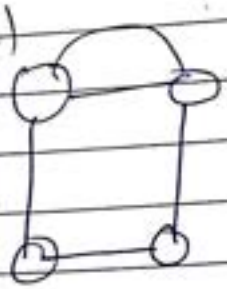
1) $O(n^2 \log n)$ $O(n!)$

SP — LP

Shortest Path Longest Path

2) Euler's tour $O(E)$

every edge exactly once but vertices can be repeated

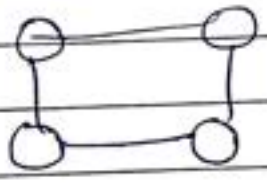


ETour.



Hamiltonian cycle (Visiting every vertex once & come to source vertex again)

$O(2^n)$



3) 2CNF — 3CNF — CNF → Conjunctive Normal form

↓ ↓ and

$(x_1 \vee x_2) \wedge (x_3 \vee x_4)$ Polynomial Don't know

or And

Optimization Problem

Travelling Salesman Problem (TSP). ∆ graph G, 331
1) shortest path covering all vertices exactly once

2) 0/1 KS: Given C, P, W. find max profit

3) LCS: A, B Find LCS

We want to see above are tractable or intractable

So we take easier problems than above 3 and find them and if they are not solvable then these are Intractable Decision Problem (for every OP) (hard)

So 1) Is there any SP covering all vertices of length atmost K (Yes/No)

2) Is there any Sol^n whose profit is atleast 'K' (Yes/No)

3) Is there any SS whose length is atleast 'K' (Yes/No)

Opt → DP

If opt is $O(n^k)$

Then DP is ''

If DP is hard. ($\Sigma O(n^{\log n})$)

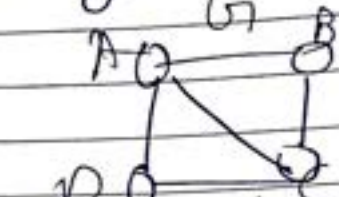
then Opt is hard

Scanned with CamScanner

In this theory of NP completeness we only talk about DP (not OPT)

Verification algorithm:

Assume G



DP: Is this graph Hamiltonian?
Ans A-B-C-D-A

Now, Verify above Ans is correct or not

Now verification algorithm is any algorithm which helps in doing so.
1) Check if all vertices present
2) Check if all edges ''

So Yes

P → Set of All DP having Polynomial time algorithm to solve them.

NP → Set of all DP for which there is a polynomial time verification (easily / fast)

P ⊂ NP

G MST almost 10
Min Spanning Tree (n)
(Prims or Kruskal's)

(Shortest) KP Profit is atleast 10
O(n log n)
(either you answer yes or no fast is it is Polynomial with loops count)

Sorting, Searching, etc.

TSP
O/1 KP

Scanned with CamScanner

∴ we have verification algorithm

Note - Every P problem falls in NP and also NP has our empty set of problem



NP = P → Intractable
↳ Empty

P - easily solved
NP - easily verified

Polynomial time reductions

A problem 'A' is said to be polynomial time reducible to a problem 'B' if
(i) Every instance 'α' of 'A' can be transformed to some instance 'β' of B in polynomial time
(ii) Answer to 'α' is 'YES' if and only if answer to 'β' is 'YES'.

→ If 'B' is easy, then A is also easy.
→ If 'B' is in 'P' then A is also in 'P'
→ If A is not in P then B is not in P

Eg A. Given 'n' boolean variables with values $x_1, x_2 \dots x_n$ does at least one variable has the value "True"?
B. Given 'n' integers $i_1, i_2 \dots i_n$ is $\max(i_1, i_2 \dots i_n) > 0$

We say A = Y (T, F, F, T) (YES)
(-30, 10, 0, 2) (YES)
B → O(n) → Polynomial

Scanned with CamScanner

Imp → Both A & B should be Decision Problem

Now let's Reduce A → B

(T, F, F, T) A

(1, 0, 0, 1) B

And yes in O(n) we are able to convert it



Let say we are able to convert every subproblem in O(Polynomial) to A

Benefits

1) Makes everything easy

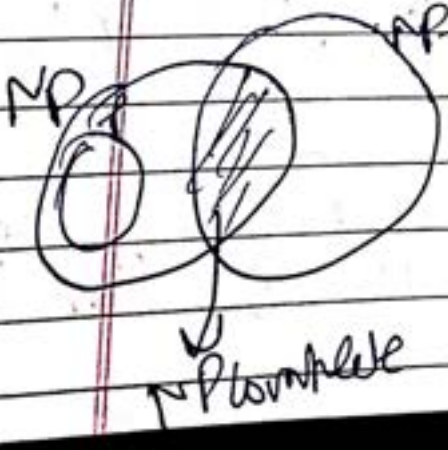
If we are able to convert every subproblem to A & A solves in O(Polynom)

NP-hard: If every problem in NP can be polynomial time reducible to a problem 'A', then 'A' is called NP-hard.

If 'A' could be solved in $O(n^k)$ then every problem in NP is 'P'.

and (P = NP)

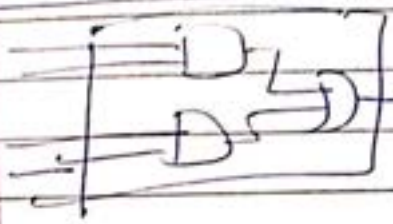
NP-complete → If A is inside NP circle then it is called NP-hard as well as NP complete.



Scanned with CamScanner

→ NP Hard & NP-complete is 3 SAT
Polynomial time, then NP=P
→ NP problem or NP-complete problems
proven to be not solvable in
polynomial time. Then P≠NP

→ A is NP-Hard, and A ≤p B
then B is NP-hard
→ and if B is in NP then B is NP complete



Is there any comb^n of circuit which will give true at end

NP (circuit-Sat)

(NP hard) Sat → formula like (x+y+3).(xy)(y3) etc.

NP hard 3CNF SAT

NP complete Subset Sum

, (x+y+3)(y+3+x)

NP complete clique

plus

vertex cover

plus

H+M+C+L

TSP

→ Traveller Salesman Problem

At how many vertex we put security guard At all edges are covered

Scanned with CamScanner

2013

Q Ram & Shyam have been asked to show that a certain problem Π is NP complete. RAM shows a polynomial time reduction from 3-SAT problem to Π, and Shyam shows a polynomial time reduction from Π to 3-SAT. What is Π

Ans - Π is NP-hard (not NP Complete)

Q. Problem 3-SAT and 2-SAT are,

Ans (a+b+c), (a+b).

Q Let 'S' be an NP complete problem and 'Q' and 'R' be 2 other problems not known to be in NP. 'Q' is Polynomial time reducible to 'S' and 'S' is Polynomial-time reducible to R.

Ans Q → S → R (NP-hard)

Nothing can be said

Q Let A be a problem that belongs to the class NP. Then which of the following is true

a) There is no polynomial time algorithm for 'A'.

b) If 'A' can be solved deterministically in polynomial time, then, P=NP

Scanned with CamScanner

c) If 'A' is NP-hard then it is NP-complete
d) A may be undecidable
∴ C : O(E)

Q. Which is true
1) The problem of determining whether there exists a cycle in an undirected graph is in 'P'

2) The problem of determining whether there exists a cycle in an undirected graph is 'NP'
3) If a problem A is NP-complete there exists a non deterministic polynomial time algorithm to solve A.

True became

P → Deterministic time (Turing M/C)
NP - Non (... (So true)

If NP instead of NP complete then also (Turing M/C)

Q. Which of the following is not NP-hard?
a) Hamiltonian circuit problem
b) 0/1 knapsack problem
c) finding bi-connected components of a graph
d) The graph colouring problem.

Scanned with CamScanner

Q Let SHAM$_3$ be a problem of finding hamiltonian cycle in a graph $G=(V,E)$ with $|V|$ divisible by 3 & DHAM$_3$ be a problem of determining if Hamiltonian cycle exists in such graphs. Which of the following is true?

Ans We know ~~SHAM$_3$~~ HC — NP complete

DHAM — NP Complete

SHAM → $|V| = 0 \bmod 3$

If DHAM $\xrightarrow{\text{"P"}}$ SHAM then SHAM is NP-hard & NP-complete

Remainder 0 → Straight conversion → 0

1 eg 4 vertices then add 2 more then 6

2 → add only 1 vertex

So DHAM $\xrightarrow{(P)}$ SHAM

∴ 1 or 2 vertex added So NP-Complete NP-Hard

Q- Suppose a polynomial time algorithm is discovered that correctly computes the largest clique in a given graph. In this scenario, which one of the following represents the correct Venn diagram of the complexity classes P, NP & NPC?

(a) NP, P, NPC

(b) P, NP, NPC

(c) P, NP, NPC

(d) P = NP = NPC

SAT — Satisfiability

Page 339

Am-1 P-S it's optimization problem & not decision problem.

NP=P : Polynomial time algo is discovered for NP (Largest Clique, ~~Longest Path~~, a, b, in Level)

Am (d) → : If every NP problem is NP hard and = NPC it would have been earlier why we have not changed NPC definition

Q Consider the decision problem 2CNF-SAT
(a) NP-complete
(b) Solvable in polynomial time by reduction to directed graph reachability
(c) Solvable in constant time since any I/P instance is satisfiable
(d) NP-hard, but not NP

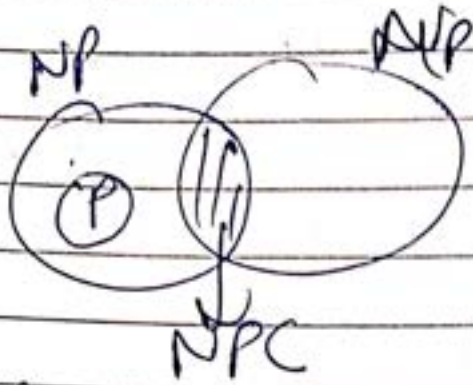
Am 2CNF (a∨b)∧(c̄∨b) etc

NP I/P → P' → P' = Q. If an NP-hard problem can be solved in polynomial time then P=NP

NP reducible to NP hard solved using polynomial → NP=P

Why we focus more on NP-complete than NP-hard

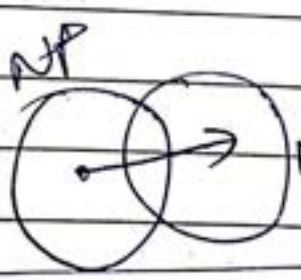
Ans =



- See if we take NPC & we prove polynomial then P = NP
  → If not then P ≠ NP
  → But if we take NPH we can't derive any relation if not.

Q1 The set of NP problems is not closed under polynomial time reductions (T/F);

Ans →



Scanned with CamScanner